Symbolic and Distributed Artificial Intelligence

Course Notes

Contents

I Introduction **5**

1 Introduction to Artificial Intelligence **5**

1.1 About the course . . . 5
1.2 What is intelligence? What is AI? . . . 5
1.2.1 The word and a classic definition . . . 6
1.2.2 Many kinds of intelligence . . . 6
1.2.3 Thinking, fast and slow: System 1 and System 2 . . . 7
1.2.4 Artificial intelligence and intelligent agents . . . 8
1.3 A short history of AI . . . 9
1.3.1 Before computers . . . 10
1.3.2 1950: the Turing test . . . 11
1.3.3 1956: the Dartmouth workshop and the founding fathers . . . 12
1.3.4 1966–1972: Shakey the robot . . . 12
1.3.5 1970–1972: Prolog . . . 13
1.3.6 The AI winters and the Lisp machines . . . 13
1.3.7 1997–2016: from chess to Go, the data-driven era . . . 14
1.4 Some more recent achievements . . . 15
1.4.1 Games, medicine and emergent behaviour . . . 15
1.4.2 Self-driving cars: the SAE levels of driving automation . . . 16
1.4.3 Generative AI, protein folding and agentic AI . . . 16
1.5 But... risks, and the answers to them . . . 17
1.5.1 Things that went wrong, or might . . . 17
1.5.2 And hence: principles and guidelines . . . 19
1.6 Symbolic vs. sub-symbolic AI, and what this course is about . . . 19
1.6.1 Two families of AI . . . 19
1.6.2 Thinking fast and slow in AI . . . 20
1.6.3 What SDAI is about . . . 21

II Logic **23**

2 Propositional logic **23**
2.1 What is a logic? . . . 23
2.2 Propositions and connectives . . . 24
2.2.1 Propositions . . . 24
2.2.2 The connectives . . . 25
2.2.3 Building complex formulas . . . 27
2.3 Valuations, tautologies and consistency . . . 28
2.3.1 Valuations: giving a truth value to a formula . . . 28
2.3.2 Tautologies, consistency, inconsistency . . . 29
2.3.3 Logical equivalence and logical consequence . . . 30
2.4 Proof theory: natural deduction . . . 31
2.4.1 What is a proof? . . . 31
2.4.2 Basic rules without subproofs . . . 32
2.4.3 Rules with subproofs . . . 36
2.4.4 Derived rules . . . 43

3 First-order logic **50**
3.1 Why not propositional logic? . . . 51
3.2 Syntax of FOL: terms and predicates . . . 52
3.2.1 Terms: constants, variables, functions . . . 52
3.2.2 Predicates . . . 53
3.3 Quantifiers . . . 55
3.3.1 The three quantifiers and their syntax . . . 56
3.3.2 Free and bound variables . . . 57
3.3.3 Quantifiers as big conjunctions and disjunctions . . . 57
3.4 Decidability . . . 59
3.5 Proof theory for first-order logic . . . 60
3.5.1 Frege (Hilbert-style) proofs . . . 60
3.5.2 Provability and soundness . . . 62
3.5.3 Sequent calculus LK . . . 63
3.5.4 LKe: adding equality . . . 64
3.5.5 Natural deduction rules for quantifiers . . . 64

III Agents and Multi-Agent Systems **65**

4 Agents and multi-agent systems **66**
4.1 What is an agent? . . . 66
4.1.1 The weak definition . . . 66
4.1.2 The strong definition . . . 68
4.2 What is a multi-agent system? . . . 68
4.3 Three ways of using MAS . . . 70
4.3.1 MAS as a software engineering metaphor . . . 70

4.3.2 Agent-based development 70
4.3.3 MAS for simulation 72
4.4 A short history of agents and MAS 72
4.4.1 From artificial intelligence 73
4.4.2 From object-oriented programming 73
4.4.3 From human-computer interaction 74
4.4.4 Agents as intentional systems 75
4.4.5 Two ancestors of MAS: Contract Net and Actors 76

5 Agent architectures **79**

6 Engineering multi-agent systems **79**

IV Prolog **79**

7 Prolog **79**

V Cognitive Agents, Communication and Platforms **79**

8 Cognitive agents: BDI and AgentSpeak(L) **79**

9 Agent communication **79**

10 JADE: a platform for multi-agent systems **79**

11 Agent-based modelling and simulation **79**

List of whiteboards

1 Terms and atoms (written after the FOL lectures) 54
2 Modus Ponens: Socrates is mortal (written during the FOL proof-theory lecture) 61

The professor wrote these on the whiteboard during the lectures. They are exam material: each one is redrawn in a clean form, and the original photo is shown next to it.

Part I

Introduction

1 Introduction to Artificial Intelligence

[Slides · Thinking machines: 1–97]

Before studying logic, Prolog and agents, the course opens with a big-picture tour: what *intelligence* is, how *artificial* intelligence (AI) was born and grew, what it can do today, why people are worried about it, and where this course sits in the AI landscape. This chapter covers:

1. what intelligence is (several definitions, Kahneman's System 1 and System 2);
2. what artificial intelligence is, and why it is defined through *agents*;
3. a short history of AI, from Aristotle to self-driving cars;
4. recent achievements (AlphaStar, AlphaFold, generative and agentic AI);
5. the risks, and the principles and rules written to control them;
6. symbolic vs. sub-symbolic AI, and why this course is about the *symbolic* and *distributed* side.

1.1 About the course

[Slides · Thinking machines: 1–3]

The course **Symbolic and Distributed Artificial Intelligence** (SDAI, sometimes written S&DAI) is taught by prof. Viviana Mascardi at the University of Genoa. All material (slides, announcements, lab instructions) is published on the course page of the university e-learning platform (AulaWeb).

The course has two souls, already visible in its name:

- **Symbolic**: intelligence obtained by manipulating *symbols* with explicit rules: logic (Part II of these notes) and the logic programming language **Prolog** (Part IV);
- **Distributed**: intelligence spread over many autonomous software **agents** that interact: agents, multi-agent systems, their architectures, their communication and the platforms used to build them (Parts III and V).

Section 1.6 explains, at the end of this chapter, why these two things belong together.

1.2 What is intelligence? What is AI?

[Slides · Thinking machines: 4–12]

To build *artificial* intelligence we should first agree on what intelligence is. There is no single accepted answer; the slides show several complementary views.

1.2.1 The word and a classic definition

[Slides · Thinking machines: 4–6]

The word comes from Latin *intelligĕre*: "to understand", literally "to read inside" (*intus legere*) or "to read between" (*inter legere*), that is, to read beyond the surface of things.

Definition — Intelligence (Mainstream Science on Intelligence, 1994)

Intelligence is a very general mental capability that, among other things, involves the ability to **reason**, **plan**, **solve problems**, **think abstractly**, **comprehend complex ideas**, **learn quickly** and **learn from experience**. It is not merely book-learning, a narrow academic skill, or test-taking smarts. Rather, it reflects a broader and deeper capability for comprehending our surroundings: "catching on", "making sense" of things, or "figuring out" what to do.

The definition was published in the *Wall Street Journal* in December 1994 and signed by 52 scientists. The professor remarks that the statement also contains claims she does not feel comfortable with: it is a reference point, not an undisputed truth. The seven abilities in bold (highlighted in red on slide 6) are worth remembering: each of them has become a branch of AI.

Ability in the definition	AI field that tries to reproduce it
reason	logic, automated reasoning, Prolog (this course)
plan	automated planning (e.g. the STRIPS planner of Shakey)
solve problems	search algorithms, constraint solving
think abstractly	knowledge representation, ontologies
comprehend complex ideas	natural language understanding
learn quickly / from experience	machine learning, reinforcement learning

Table 1: The abilities in the 1994 definition and the AI research areas that study them.

1.2.2 Many kinds of intelligence

[Slides · Thinking machines: 7–8]

Intelligence is not a single number. Two influential theories:

Definition — Multiple intelligences (H. Gardner, *Frames of Mind*, 1983)

Gardner proposed that humans have several distinct intelligences:

1. musical-rhythmic and harmonic;
2. visual-spatial;
3. verbal-linguistic;
4. logical-mathematical;
5. bodily-kinesthetic;
6. interpersonal (understanding others);
7. intrapersonal (understanding oneself);
8. naturalistic (recognising and classifying nature);
9. existential (asking deep questions about life).

Definition — Emotional intelligence (D. Goleman, 1995)

Emotional intelligence (EI) is the capability of individuals to recognise their own emotions and those of others, discern between different feelings and label them appropriately, use emotional information to guide thinking and behaviour, and manage and/or adjust emotions to adapt to environments or achieve one's goal(s).

Intuition — Why does an AI course care?

If intelligence has many faces, then "artificial intelligence" can also mean many different things. A chess program shows logical-mathematical intelligence; a robot that walks on rough ground shows bodily-kinesthetic intelligence; a chatbot that adapts its tone to an angry customer imitates a bit of emotional and interpersonal intelligence. A *multi-agent system* (the main topic of this course) needs *interpersonal* (social) intelligence: agents must understand and negotiate with each other.

1.2.3 Thinking, fast and slow: System 1 and System 2

[Slides · Thinking machines: 9–11]

In the book *Thinking, Fast and Slow* (2011), the psychologist and Nobel prize winner Daniel Kahneman describes the mind as two cooperating "systems".

Definition — System 1 and System 2

- **System 1**: fast, automatic, frequent, emotional, stereotypic, unconscious.
- **System 2**: slow, effortful, infrequent, logical, calculating, conscious.

System 1 can… (in order of complexity)	**System 2 is needed to…**
determine that an object is farther than another	prepare for the start of a sprint
localise the source of a sound	direct your attention to the clowns at the circus
complete the phrase "war and …"	look for the woman with grey hair
show disgust at a gruesome image	try to recognise a sound
solve $2 + 2 =$?	sustain a faster-than-normal walking rate
read text on a billboard	judge if a behaviour is appropriate in a social setting
drive a car on an empty road	count the number of A's in a text
think of a good chess move (if you are a chess master)	give someone your phone number
understand simple sentences	park into a tight parking space
	compare the price/quality ratio of two washing machines
	determine the validity of a complex logical reasoning

Table 2: Kahneman's examples (slides 10–11). Notice that the same task can move from System 2 to System 1 with practice: a chess master "sees" a good move.

Worked example — the bat and the ball

[Not in the slides: added material]

A bat and a ball cost 1.10 euro in total. The bat costs 1 euro more than the ball. How much does the ball cost? System 1 shouts "10 cents!". System 2, if you wake it up, checks: if the ball costs x, the bat costs $x + 1$, so

$$\begin{aligned} x + (x + 1) &= 1.10 \\ 2x &= 0.10 \\ x &= 0.05. \end{aligned}$$

The ball costs 5 cents (check: $0.05 + 1.05 = 1.10$). Fast intuition is cheap but can be wrong; slow reasoning is expensive but verifiable. Keep this in mind: it is exactly the difference between *sub-symbolic* and *symbolic* AI (Section 1.6).

1.2.4 Artificial intelligence and intelligent agents

[Slides · Thinking machines: 12]

Definition — Artificial intelligence

Artificial intelligence (AI) is intelligence demonstrated by machines, unlike the natural intelligence displayed by humans and animals. The leading AI textbook, *Artificial Intelligence: A Modern Approach* (S. Russell and P. Norvig, 4th edition),

defines the field as the study of **intelligent agents**: any device that **perceives** its environment and **takes actions** that maximise its chance of successfully achieving its **goals**.

This definition is the bridge to the whole course: AI is not defined by *how* the device works inside (neural network, logic rules, ...) but by its behaviour as an **agent**, a loop of perceiving and acting towards goals.

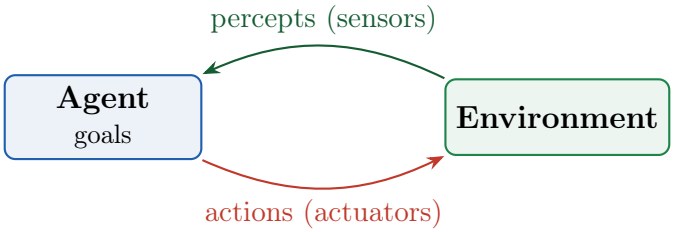


Figure 1: An intelligent agent: it perceives the environment and acts on it to reach its goals. Part III of these notes is entirely about this picture.

Why this matters in practice

Everyday intelligent agents: a robot vacuum cleaner (perceives obstacles and dirt, acts by moving and sucking, goal: a clean floor); a smart thermostat (perceives temperature and your habits, acts on the boiler, goal: comfort at minimum cost); a trading bot (perceives prices, acts by buying/selling, goal: profit); a navigation app (perceives traffic, acts by suggesting a route, goal: get you there fast).

1.3 A short history of AI

[Slides · Thinking machines: 13–67]

The slides build a timeline one event at a time (slides 16–67). The complete timeline is redrawn in Figure 2; the sub-sections below explain each step.

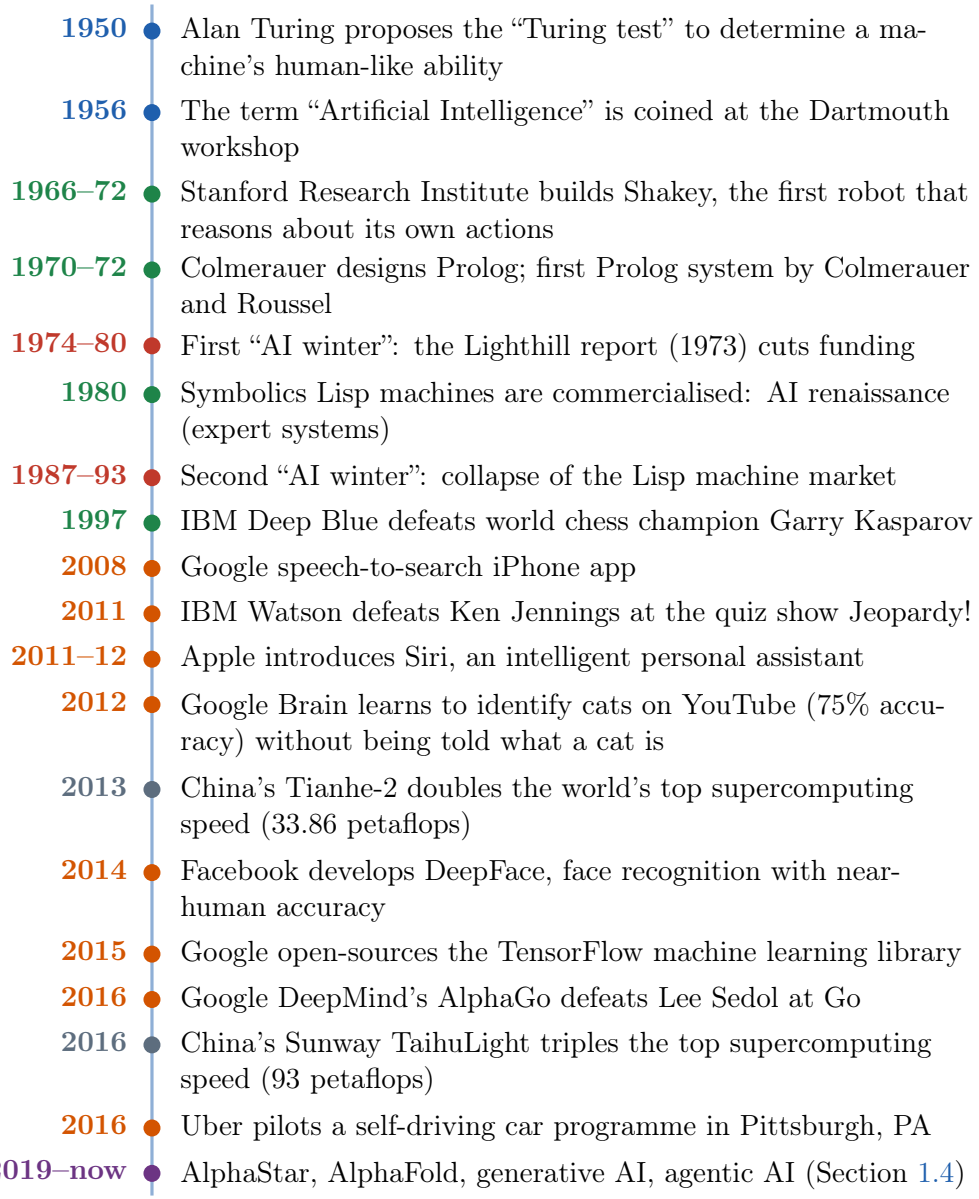


Figure 2: The AI timeline of the slides (slide 67, extended). Colours: foundations, symbolic AI successes, AI winters, data-driven / neural successes, computing power, today. Notice the shift: until the 1990s the big results are symbolic (reasoning, planning, chess search); after 2008 they are mostly learned from data.

1.3.1 Before computers

[Slides · Thinking machines: 13–15]

The slides show three portraits, without text. Here is why each person is on the "before" list (the explanations are added):

- **Aristotle** (384–322 BC) invented the first system of formal logic, the *syllogism*: "All men are mortal; Socrates is a man; therefore Socrates is mortal". The very

same example will come back in first-order logic (Whiteboard 2) and in Prolog.

- **Gottfried Wilhelm von Leibniz** (1646–1716) dreamed of a universal symbolic language (*characteristica universalis*) and a "calculus of reasoning" (*calculus ratiocinator*), so that disputes could be settled by saying "let us calculate!". He also studied binary arithmetic and built a mechanical calculator. This is the dream of symbolic AI: reasoning as computation.
- **Mary Shelley** (1797–1851) wrote *Frankenstein* (1818), the story of an artificial being created by a scientist, and of the consequences of creating it: the first great story about the risks of artificial creatures (see Section 1.5).

1.3.2 1950: the Turing test

[Slides · Thinking machines: 16–23]

In 1950 Alan Turing published the paper *Computing Machinery and Intelligence* in the philosophy journal *Mind*. Instead of answering the vague question "Can machines think?", he replaced it with a game that can actually be played, the **imitation game**, now called the **Turing test**.

Definition — Turing test

A human interrogator chats (in writing) with two hidden partners: a human and an AI system. By asking questions, the interrogator must decide which one is the machine. If the interrogator cannot reliably tell them apart, the machine passes the test.

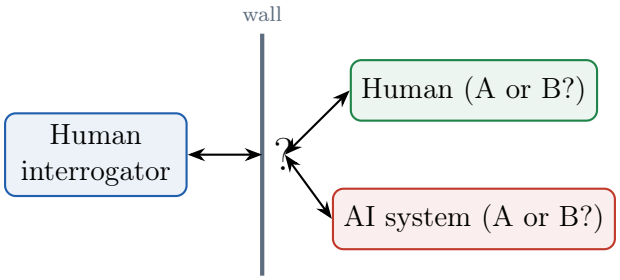


Figure 3: The imitation game (slide 22): the interrogator only sees written answers and must guess which partner is the machine.

Intuition — Behaviour instead of definitions

Turing's trick is to judge intelligence by *observable behaviour*, not by what happens inside. The agent definition of AI (Section 1.2.4) has the same spirit. The slides add a comic ("Turing test 2208") where the human fails: "the machine realised immediately that it was me being the dumb human". Today large language models make the classic Turing test much less decisive than it looked in 1950.

An open parenthesis (slides 19–21). What does it exactly mean to "think", to "perceive", to be "conscious of being alive"? The slides link to the neuroscientist Anil

Seth (TED talk *Your brain hallucinates your conscious reality*): according to him, perception is a *controlled hallucination*, the brain's best guess about the causes of its sensory signals, constantly corrected by the senses. If we cannot define these words precisely for humans, judging them in machines is even harder. (Closed parenthesis.)

1.3.3 1956: the Dartmouth workshop and the founding fathers

[Slides · Thinking machines: 24–33]

In summer 1956 a workshop at Dartmouth College (USA) coined the term **"Artificial Intelligence"**. Its proposal contained the founding conjecture of the field:

> *"Every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it."*

The slides list the participants (the "founding fathers of AI") and, step by step, what each of them is remembered for:

Person	**Known for**
John McCarthy	the *Advice Taker* (a program reasoning with logical sentences, which inspired Prolog), the Lisp language, ALGOL
Marvin Minsky	artificial neural networks, the *society of mind* (the mind as a society of simple interacting agents: an ancestor of multi-agent systems!)
Claude Shannon	information theory, cryptography, natural language processing
Ray Solomonoff	machine learning, prediction, Bayesian frameworks
Allen Newell & Herbert Simon	the *Logic Theory Machine* (proved theorems of *Principia Mathematica*) and the *General Problem Solver*
Arthur Samuel	search, alpha-beta pruning, machine learning (a checkers program that learned to play better than its author)
Oliver Selfridge	Pandemonium theory (pattern recognition by many competing "demons")
Nathaniel Rochester	IBM, designer of the IBM 701 computer
Trenchard More	natural deduction (the proof system of Section 2.4)

Table 3: The founding fathers of AI and their contributions (slides 26–33).

Intuition — Symbolic and sub-symbolic, from day one

Already at Dartmouth both souls of AI were present: logic and symbols (McCarthy, Newell and Simon, More) and learning and neural networks (Minsky, Solomonoff, Samuel). The rest of the history is a pendulum between the two.

1.3.4 1966–1972: Shakey the robot

[Slides · Thinking machines: 34–36]

Shakey was the first general-purpose mobile robot able to *reason about its own actions*. Other robots had to be instructed on each individual step; Shakey could analyse a command and break it down into basic chunks by itself. The project combined robotics, computer vision and natural language processing, and was the first to meld *logical reasoning* and *physical action*. It was developed at the AI Center of the Stanford Research Institute (now SRI International) from about 1966 to 1972 (project manager Charles Rosen), programmed mainly in LISP, and its planning component was the **STRIPS** planner. As the first logical, **goal-based agent**, Shakey lived in a limited world (a few rooms with boxes).

Worked example — what "breaking a command into chunks" means

[Not in the slides: added material]

Command: "push the box from room A to room B". A planner like STRIPS knows actions with preconditions and effects, e.g. `goto(Place)`, `push(Box, From, To)`, and finds the sequence: `goto(room_A)` → `goto(box)` → `push(box, room_A, door)` → `push(box, door, room_B)`. Your navigation app and a warehouse robot do the same kind of planning today. Shakey is a deliberative agent in the sense of Section 5.

1.3.5 1970–1972: Prolog

[Slides · Thinking machines: 37]

In 1970 Alain Colmerauer designed the **Prolog** language (from the French *programmation en logique*); the first Prolog system was implemented by Colmerauer and Philippe Roussel in 1972 in Marseille. Prolog is the language of Part IV of these notes: a program is a set of logical facts and rules, and running it means proving a query.

1.3.6 The AI winters and the Lisp machines

[Slides · Thinking machines: 38–43]

Early AI promised a lot. When the promises were not kept, funding collapsed: these periods are called **AI winters**.

- **First AI winter (1974–1980).** The **Lighthill report** (*Artificial Intelligence: A General Survey*, Sir James Lighthill, 1973, for the British Science Research Council) gave a very pessimistic evaluation: "in no part of the field have discoveries made so far produced the major impact that was then promised". It formed the basis for the British government's decision to end support for AI research in all but three universities (Edinburgh, Sussex, Essex). Its main technical criticism: AI had failed to address the **combinatorial explosion** that appears when problems are taken from toy worlds to real-world domains.
- **AI renaissance (1980).** Symbolics commercialises **Lisp machines**: computers with hardware support for Lisp operations (data type testing, CDR coding,

incremental garbage collection), running large Lisp programs very efficiently, with operating systems written entirely in Lisp. They powered the boom of *expert systems* (rule-based programs encoding the knowledge of human experts).

- **Second AI winter (1987–1993).** With the microcomputer revolution, cheap desktop PCs soon ran Lisp programs even faster than Lisp machines, without special hardware. The Lisp machine market collapsed: most makers went out of business by the early 1990s, and AI lost credibility again.

Worked example — combinatorial explosion in numbers

[Not in the slides: added material]

Chess has about 35 legal moves per position. Looking d moves ahead means examining about 35^d positions:

$$35^2 = 1\,225, \qquad 35^4 = 1\,500\,625, \qquad 35^6 \approx 1.8 \times 10^9, \qquad 35^{10} \approx 2.8 \times 10^{15}.$$

Each extra level multiplies the work by 35. An algorithm that works on a toy problem can become hopeless on a slightly bigger one: this is what the Lighthill report meant. Deep Blue (1997) beat it on chess with special hardware plus pruning (alpha-beta, Samuel's legacy).

1.3.7 1997–2016: from chess to Go, the data-driven era

[Slides · Thinking machines: 44–67]

- **1997**: IBM **Deep Blue** defeats the world chess champion Garry Kasparov (symbolic AI: search plus hand-written evaluation).
- **2008**: Google releases a **speech-to-search** iPhone app: you speak, the recording is sent to Google's servers, and the results come back in seconds.
- **2011**: IBM **Watson** defeats the champion Ken Jennings at the quiz show *Jeopardy!* (natural language question answering).
- **2011/2012**: Apple introduces **Siri**, an intelligent personal assistant.
- **2012**: after seeing 10 million images from YouTube videos in three days, the 16,000-computer network of **Google Brain** (one billion connections) began to recognise cats, even though it had never been taught what a cat looked like (unsupervised learning, 75% accuracy).
- **2013**: China's Tianhe-2 doubles the world's top supercomputing speed (33.86 petaflops); **2016**: Sunway TaihuLight triples it (93 petaflops). Raw computing power is one of the reasons for the success of deep learning.
- **2014**: Facebook's **DeepFace** identifies human faces in images: a nine-layer neural network with over 120 million connection weights, trained on four million images uploaded by Facebook users, said to be 97% accurate (vs. 85% for the FBI's Next Generation Identification system). It uses a 3D model to virtually rotate faces so that they look at the camera.
- **2015**: Google open-sources **TensorFlow**, a library for numerical computation with data flow graphs, now widely used for machine learning.

- **2016**: Google DeepMind's **AlphaGo** defeats Lee Sedol at Go, a game whose search space is far too large for Deep Blue-style brute force.
- **2016**: Uber pilots a **self-driving car** programme in Pittsburgh, PA.

Watch out — Two kinds of "winning"

Deep Blue and AlphaGo both beat a world champion, but in very different ways. Deep Blue *searched* millions of positions per second with rules written by humans (System 2 style, symbolic). AlphaGo *learned* an intuition for good moves from millions of games with neural networks, combined with a search (System 1 + System 2). Do not file both under "AI wins at games" without noting the difference: it is the theme of Section 1.6.

1.4 Some more recent achievements

[Slides · Thinking machines: 68–79]

After 2016 the pace accelerated. The slides show a selection of results; for each one, here is what it is and why it matters.

1.4.1 Games, medicine and emergent behaviour

[Slides · Thinking machines: 68–71]

- **AlphaStar** (DeepMind, 2019). StarCraft II is a real-time strategy video game: many units, partial information (you do not see the whole map), real-time decisions. AlphaStar won its historic series against professional players (announced on January 24, 2019, after matches recorded in December 2018; e.g. 5–0 against the pro player MaNa).
- **Lung cancer screening** (*Nature Medicine*, May 2019). A three-dimensional deep learning model analyses low-dose chest CT scans end to end and detects lung cancer with performance comparable to, or better than, expert radiologists.
- **Emergent tool use from multi-agent autocurricula** (OpenAI, ICLR 2020). Agents play *hide-and-seek* in a simulated world with movable boxes and ramps. Nobody tells them how to use the objects; just by competing, the teams discover a sequence of strategies: hiders build shelters with boxes, seekers use ramps to jump in, hiders then lock the ramps away, seekers "surf" on boxes, and so on.

Why this matters in practice — A multi-agent system in action

The hide-and-seek experiment is a *multi-agent system*: several autonomous agents, in a shared environment, competing (hiders vs. seekers) and cooperating (inside a team). Complex behaviour *emerges* from simple goals and interaction, with no central controller. Keep this example in mind for Part III (agents and MAS) and for agent-based simulation (Section 11).

1.4.2 Self-driving cars: the SAE levels of driving automation

[Slides · Thinking machines: 72–74]

How "autonomous" is an autonomous car? The SAE J3016 standard defines six levels.

Level	What the human must do	What the features do	Examples
0	You **are driving**, even if features are on; constantly supervise	warnings and momentary assistance	automatic emergency braking, blind-spot warning, lane departure warning
1	same as level 0	steering **or** brake/acceleration support	lane centering **or** adaptive cruise control
2	same as level 0	steering **and** brake/acceleration support	lane centering **and** adaptive cruise control together
3	You are **not driving** while features are engaged, but you must take over when the system requests it	drives the vehicle under limited conditions	traffic jam chauffeur
4	You are not driving; you will never be required to take over	drives under limited conditions, never needs you	local driverless taxi; pedals and steering wheel may not be installed
5	You are not driving	drives everywhere, in all conditions	as level 4, but anywhere

Table 4: SAE levels of driving automation (slide 72). Blue rows: *driver support* features (the human drives). Green rows: *automated driving* features (the system drives).

The slides note (2024 news) that a **Level 3** car can now be bought in the US: the Mercedes-Benz system works only on a small selection of roads, at limited speed and only in some states, and its wide adoption will require regulators to catch up.

Intuition — Levels of autonomy, again later

The question "how much can the system decide by itself, and when must a human step in?" is the question of **autonomy**, the defining property of agents (Section 4). The jump from level 2 to level 3 is exactly the moment in which responsibility moves from the human to the agent.

1.4.3 Generative AI, protein folding and agentic AI

[Slides · Thinking machines: 75–79]

On slide 75 the professor asked ChatGPT itself: "which are the three most impressive achievements in the AI research field in the last three years (short answer)". The answer:

1. **Generative AI**: models such as GPT-4 and DALL-E generate coherent text, images and creative content.
2. **AI in protein folding**: DeepMind's AlphaFold accurately predicts protein structures, solving a 50-year-old problem in biology.
3. **AI alignment and safety research**: progress in interpretability, alignment and safety (e.g. by OpenAI and Anthropic) to control the behaviour of advanced AI.

Definition — Protein folding problem

A protein is a chain of amino acids. The **protein folding problem** is the question of how a protein's amino acid sequence dictates its three-dimensional atomic structure. It first emerged around 1960, with the first atomic-resolution protein structures.

AlphaFold2 (DeepMind) predicts the 3D structure of proteins from their amino acid sequence with near-experimental accuracy; more than 200 million predicted structures have been released. Knowing a protein's shape tells biologists how it works, which speeds up drug discovery and protein design (2023 review; PNAS 2024, "AlphaFold two years on").

Finally, slide 79: **agentic AI**, presented as "the next evolution of artificial intelligence". An agentic AI system does not just answer one question: given a goal, it plans, calls tools (search engines, APIs, other programs), observes the results and acts again, autonomously, over many steps.

Why this matters in practice — Old ideas, new engines

Agentic AI is the 2020s name for an idea this course has studied for decades: **autonomous agents** that perceive, decide and act towards goals, and **multi-agent systems** in which several of them coordinate. The difference is the engine: a large language model instead of (or together with) explicit beliefs, goals and plans. Understanding classic agents (BDI, communication protocols, platforms) is what makes it possible to engineer agentic AI systems that are reliable.

1.5 But... risks, and the answers to them

[Slides · Thinking machines: 80–87]

The slides now switch tone ("But..."): AI systems already take or support decisions that affect people, and things can go wrong.

1.5.1 Things that went wrong, or might

[Slides · Thinking machines: 80–84]

Case	What happened	The underlying risk
State v. Loomis (2016–17)	The Wisconsin Supreme Court allowed judges to use an algorithmic risk-assessment score (predicting the risk of re-offending) in sentencing, but required a written warning about its limits. The defendant could not inspect how the proprietary algorithm worked.	opaque decisions about people's freedom; possible bias; no way to contest the reasoning
"Smart courts" in China (2019)	Courts adopt AI assistants (e.g. Shanghai's "System 206") to check evidence and support judges.	automation of justice; who is accountable?
ChatGPT banned in Italy (2023)	The Italian data protection authority temporarily blocked ChatGPT over privacy concerns (personal data used for training, no age checks).	privacy and data protection
AI leaders at the UN Security Council	The heads of OpenAI (Sam Altman) and Anthropic (Dario Amodei) gave separate briefings on AI safety to the UN Security Council.	global safety and security
Incidents with AI agents (slide 84)	News reports about security incidents involving AI platforms and AI agents that were misused or hijacked.	autonomous agents acting in the real world can be attacked or misbehave

Table 5: The cautionary examples of slides 80–84.

Intuition — Why symbolic AI helps here

In State v. Loomis the core problem is that nobody could *see* why the system gave a certain score. A symbolic system (a set of logical rules, a Prolog program, a BDI

agent with explicit plans) can *explain* its conclusions: "I concluded X because of rule R applied to facts F_1 and F_2". Explainability, verifiability and accountability are among the strongest arguments for the symbolic side of AI.

1.5.2 And hence: principles and guidelines

[Slides · Thinking machines: 85–87]

- **Asilomar AI Principles** (Future of Life Institute, 2017): 23 principles developed at the 2017 Asilomar conference, grouped into research issues, ethics and values (e.g. safety, transparency, responsibility, human control, privacy), and longer-term issues.
- **Ethics Guidelines for Trustworthy AI** (European Commission, High-Level Expert Group on AI, 8 April 2019). Trustworthy AI must be *lawful*, *ethical* and *robust*, and should meet seven key requirements (listed below, not in the slides).
- **Calls to slow down** (New York Times, September 2026): top AI leaders, among them the head of Anthropic, argued that capabilities are advancing so quickly that greater safety controls are needed across the industry.

Key result — The seven requirements for trustworthy AI (EU, 2019)

[Not in the slides: added material]

1. human agency and oversight;
2. technical robustness and safety;
3. privacy and data governance;
4. transparency (including explainability);
5. diversity, non-discrimination and fairness;
6. societal and environmental well-being;
7. accountability.

These guidelines were followed by the EU *AI Act* (in force since 2024), which regulates AI systems according to their level of risk.

1.6 Symbolic vs. sub-symbolic AI, and what this course is about

[Slides · Thinking machines: 88–97]

The deck ends by going "back to the first slides": Kahneman's two systems (Section 1.2.3) give the key to read the whole history of AI.

1.6.1 Two families of AI

[Slides · Thinking machines: 89–90]

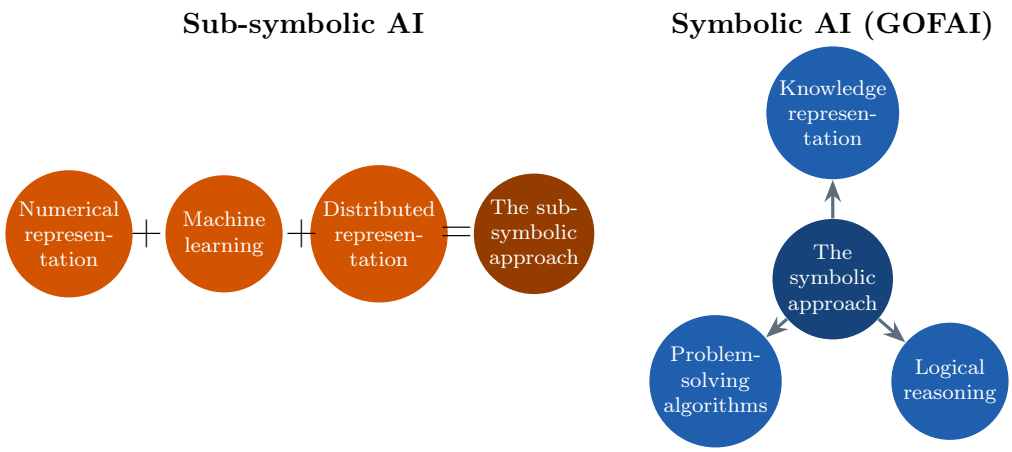


Figure 4: The two families of AI (slides 89–90). Sub-symbolic AI: knowledge is numbers spread over many parameters, learned from data. Symbolic AI: knowledge is explicit symbols, manipulated by reasoning and search.

Definition — Sub-symbolic AI

AI where knowledge is encoded as **numbers** (e.g. the weights of a neural network), **learned** from data (machine learning), and **distributed**: no single number means "cat" or "stop sign"; the meaning is spread over millions of parameters.

Definition — Symbolic AI ("Good Old-Fashioned AI", GOFAI)

AI where knowledge is written explicitly as **symbols** that humans can read (facts, rules, logical formulas, plans), and intelligence is obtained by **knowledge representation**, **logical reasoning** and **problem-solving algorithms** (search, planning).

Worked example — the same knowledge, two ways

Knowledge: "Socrates is a human; all humans are mortal".

- *Symbolic*: two lines of Prolog (Part IV), `human(socrates).` and `mortal(X) :- human(X).` Asking `?- mortal(socrates).` gives `true`, and the system can show the proof: rule + fact.
- *Sub-symbolic*: a language model has read millions of sentences about Socrates and mortality; asked "is Socrates mortal?" it answers "yes" because this is the most likely continuation. The "knowledge" is somewhere in billions of weights and cannot be pointed at.

1.6.2 Thinking fast and slow in AI

[Slides · Thinking machines: 88, 91]

The paper *Thinking Fast and Slow in AI* (G. Booch, F. Rossi and others, AAAI 2021) draws a loose parallel between Kahneman's two systems and the two main lines of work in AI:

	System 1 ≈ machine learning	System 2 ≈ symbolic AI
Style	data-driven, fast, intuitive	knowledge-driven, slow, deliberate
Good at	perception from sensory data (seeing, reading, hearing); building (possibly imprecise and biased) models from data	complex problems: logic, search, optimisation, planning, with explicit knowledge, symbols and high-level concepts
Weak at	common-sense reasoning, basic notions of causality, explanations	noisy perception, learning from raw data, scaling to huge open-ended domains
Example	recognise a pedestrian in a camera image	decide that the car must yield because traffic rules say so

Table 6: The parallel of slide 91 ("Thinking Fast and Slow in AI"), with examples.

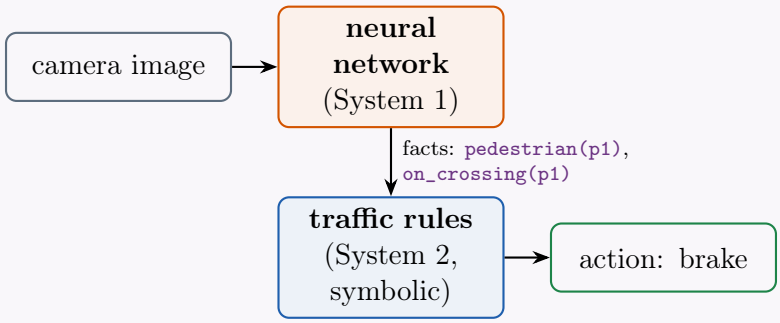
As in humans, the two sets of capabilities are **symbiotic**: this supports **hybrid, neurosymbolic** approaches to AI.

Definition — Neurosymbolic AI

Neurosymbolic AI combines neural networks (sub-symbolic learning) with symbolic representations and reasoning, to obtain systems that are more versatile and more dependable than either alone (slides 92–95: articles in *Communications of the ACM* 2022, IBM Research, the book *Neuro-Symbolic Artificial Intelligence: The State of the Art*, and the journal *Neurosymbolic Artificial Intelligence*, founded in 2023).

Worked example — a neurosymbolic self-driving car

[Not in the slides: added material]



The neural part turns pixels into *symbols* ("there is a pedestrian on the zebra crossing"); the symbolic part applies explicit, auditable rules ("if a pedestrian is on a crossing, then yield"). If the car brakes, we can explain why.

1.6.3 What SDAI is about

[Slides · Thinking machines: 96–97]

Key result — The course in one paragraph (slide 96)

The Symbolic and Distributed AI course is about the **symbolic component of neurosymbolic AI**. It exploits symbolic AI in the context of **distributed AI systems**, where (symbolic, but not only) intelligent software **agents** distributed across different machines operate **autonomously**, to **coordinate/collaborate** in order to achieve some common goal, or to **compete** to access shared resources. A system consisting of these distributed agents, as well as their environment and the organisations they are part of, is named **multi-agent system** (MAS).

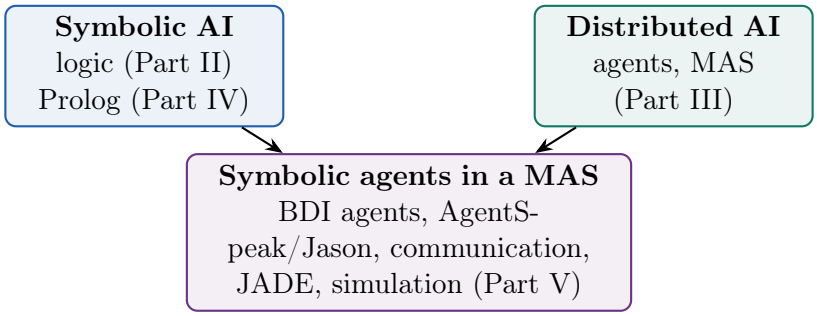


Figure 5: How the parts of these notes fit together.

Welcome on board, and let's have fun with S&DAI!

Cheat sheet — Introduction to AI

- **Intelligence**: general capability to reason, plan, solve problems, think abstractly, comprehend complex ideas, learn quickly and learn from experience. Many kinds exist (Gardner's 9 intelligences, emotional intelligence).
- **System 1** = fast, automatic, intuitive; **System 2** = slow, effortful, logical (Kahneman, 2011). Parallel in AI: System 1 ≈ machine learning, System 2 ≈ symbolic AI (logic, planning, search).
- **AI** (Russell & Norvig): the study of *intelligent agents*, devices that perceive their environment and act to maximise the chance of achieving their goals.
- **Key dates**: 1950 Turing test; 1956 Dartmouth workshop (the term "AI"); 1966–72 Shakey; 1970–72 Prolog; 1974–80 and 1987–93 AI winters; 1997 Deep Blue; 2011 Watson; 2012 Google Brain cats; 2016 AlphaGo; 2019 AlphaStar; 2020+ AlphaFold, generative AI; now agentic AI.
- **Risks**: biased decisions (State v. Loomis), privacy (ChatGPT blocked in Italy), safety and security of autonomous agents. **Responses**: Asilomar principles (2017), EU Ethics guidelines for trustworthy AI (2019).
- **Sub-symbolic AI** = numerical representation + machine learning + distributed representation. **Symbolic AI** (GOFAI) = knowledge representation + logical reasoning + problem-solving algorithms. **Neurosymbolic AI** combines them.
- **SDAI** = the symbolic component of neurosymbolic AI, used inside distributed systems of autonomous agents: a **multi-agent system (MAS)**.

Part II

Logic

2 Propositional logic

[Slides · Propositional logic: 1–21; Proof theory (prop.): 1–11]

Symbolic AI represents knowledge with *symbols* and reasons with *rules*. The mathematical tool that makes this precise is **logic**. This chapter studies the simplest logic, **propositional logic**, in three steps that will repeat for every logic of the course:

1. **syntax**: which strings of symbols are legal formulas;
2. **semantics**: what a formula means, i.e. when it is true (truth tables, valuations, tautologies);
3. **proof theory**: how to derive new formulas from old ones by applying rules mechanically (natural deduction).

The next chapter (first-order logic) repeats the same three steps for a richer language, and Prolog (Part IV) is a programming language built on a fragment of it.

2.1 What is a logic?

[Slides · Propositional logic: 1–3]

When most people say "logic" they mean either *propositional logic* or *first-order predicate logic*. In fact the word is much broader: philosophers, computer scientists and mathematicians have studied literally hundreds of logics (temporal logics, modal logics, logics of belief, ...). What do they have in common?

Definition — Logic (slide 2)

Any formal system can be considered a logic if it has:

- a well-defined **syntax**: it defines the syntactically acceptable objects of the language, called **well-formed formulae** (wff), or simply **formulae**;
- a well-defined **semantics**: it associates each formula with a **meaning**;
- a well-defined **proof theory**: it is concerned with **manipulating** formulae according to certain rules.

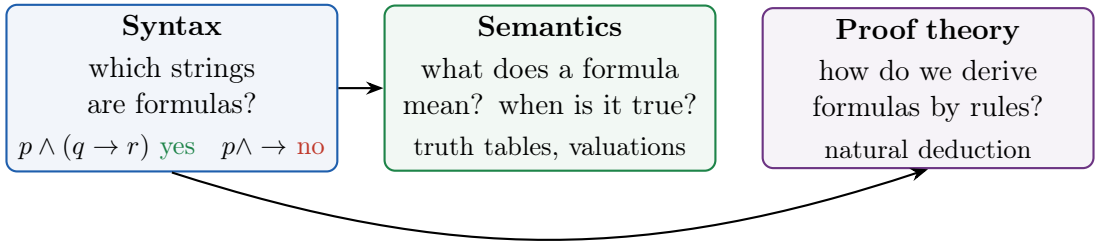


Figure 6: The three ingredients of every logic. Semantics and proof theory are two independent ways of talking about formulas: one with *truth*, one with *rules*. A good proof theory derives exactly the formulas that are semantically valid.

Intuition — A programming-language analogy

Think of a programming language. Its *syntax* says which programs compile (`x = 3 +;` does not). Its *semantics* says what a compiling program does when it runs. A *proof theory* is like a set of safe refactoring rules: you can transform the code step by step without ever running it, and the rules guarantee that the meaning is preserved. Logic works the same way, with formulas instead of programs and truth instead of behaviour.

Why this matters in practice

Every symbolic AI system makes these three choices. A Prolog program (Part IV) has a syntax (facts and rules), a semantics (what is logically implied by the program) and a proof procedure (SLD resolution) that the interpreter runs. A BDI agent (Part V) stores its beliefs as logical formulas and uses the same machinery to decide which plans apply.

2.2 Propositions and connectives

[Slides · Propositional logic: 4–18]

2.2.1 Propositions

[Slides · Propositional logic: 4–6]

Propositional logic is the simplest, and most abstract, logic we can study. Its building blocks are propositions.

Definition — Proposition (slide 4)

A **proposition** is a statement that can be either true or false; it must be one or the other, and it cannot be both.

Propositions	Not propositions
the reactor is on	are you going out somewhere? (a question)
the wing-flaps are up	$2 + 3$ (a number, not a statement)
John Major is prime minister	close the door! (an order; added)

Table 7: Examples from slide 4. (John Major was the UK prime minister when the slides were written; the sentence is still a proposition today: it is simply false.)

Key result — The "it is true that" test (slide 5)

To check whether a statement is a proposition, prefix it with *"It is true that . . . "* and see whether the result makes grammatical sense. "It is true that the reactor is on" makes sense; "It is true that are you going out somewhere?" does not.

Definition — Atomic proposition (slide 5)

An **atomic proposition** is one whose truth or falsity does not depend on the truth or falsity of any other proposition. Intuitively, atomic propositions are the *smallest* propositions: they cannot be split into smaller propositions. All the propositions above are atomic.

Instead of writing propositions in full, we abbreviate them with **propositional variables**: by convention lower-case letters $p, q, r, \ldots$. When we do so we must say what each letter means ("let p be *John Major is prime minister*"). An alternative is to use self-explaining names such as *reactor_is_on*, so that the meaning of the variable is obvious. (Prolog and AgentSpeak use exactly this second style: `reactor_on`, `door_open`.)

2.2.2 The connectives

[Slides · Propositional logic: 7–17]

Atomic propositions alone are "pretty boring". **Connectives** combine propositions into bigger, *compound* propositions. Different books use different symbols; these notes use the first column, the slides use a mix of all of them.

Symbol	Read	Name	Other notations	Arity
$\wedge$	and	conjunction	& .	binary
$\vee$	or	disjunction	| +	binary
$\neg$	not	negation	$\sim$	unary
$\rightarrow$	implies, if. . . then	implication (conditional)	$\Rightarrow$ $\supset$	binary
$\leftrightarrow$	if and only if (iff)	biconditional	$\Leftrightarrow$ $\equiv$	binary

Table 8: The connectives (slide 7). The propositional-logic slides write $\Rightarrow$ and $\Leftrightarrow$; the proof-theory slides write &, $\sim$ and $\rightarrow$. They all mean the same thing.

How do we define the meaning of a connective? With a **truth table**: a table listing the value of the compound formula for *every possible combination* of values of its components.

Key result — Number of rows of a truth table (slide 9)

If a formula contains n different atomic propositions, its truth table has 2^n rows, because each proposition can take one of 2 values (T or F) independently of the others: $\underbrace{2 \times 2 \times \cdots \times 2}_{n \text{ times}} = 2^n$. For example $n = 2 \Rightarrow 4$ rows, $n = 3 \Rightarrow 8$ rows,

$n = 10 \Rightarrow 1024$ rows.

The five truth tables. Here are the definitions of the slides, all in one table (write T for true and F for false):

p	q	$\neg p$	$p \wedge q$	$p \vee q$	$p \to q$	$p \leftrightarrow q$
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

Table 9: Truth tables of all connectives (slides 9, 11, 13, 16, 17). Orange: the two rows where $p \to q$ is true "by default" because p is false. Red: the *only* row where an implication is false.

In words:

- **And** (slides 8–9): $p \wedge q$ is true iff *both* p and q are true.
- **Or** (slides 10–11): $p \vee q$ is true iff p is true, or q is true, or both. This is the **inclusive** or.
- **Implies** (slides 12–14): $p \to q$ is true iff *either p is false or q is true*. In $p \to q$, p is the **antecedent** and q the **consequent**.
- **Iff** (slides 15–16): $p \leftrightarrow q$ is true iff p and q are both true or both false (they have the same value).
- **Not** (slide 17): $\neg p$ is true iff p is false. It is the only *unary* connective (one argument); the others are *binary*.

Watch out — Inclusive vs. exclusive "or"

In everyday language "tea or coffee?" usually means *one of the two, not both* (exclusive or). The logical $\vee$ is **inclusive**: $p \vee q$ is true also when both are true. The exclusive or (XOR) can be written $(p \vee q) \wedge \neg(p \wedge q)$, or $\neg(p \leftrightarrow q)$.

Understanding implication. The slides warn that $\to$ is the hardest connective to understand, and yet extremely important. The rule of thumb of slide 14: *$p \to q$ means "if p is true, then q is true". If p is false, we don't care about q, and by default $p \to q$ evaluates to T.*

Intuition — Implication as a promise

[Not in the slides: added material]

A parent says: "*If you pass the exam (p), I will buy you a bike (q)*". When has the parent *broken the promise*?

pass (p)	bike (q)	what happened	promise kept? ($p \to q$)
F	F	failed, no bike	T (no reason to complain)
F	T	failed, bike anyway	T (generous, but not a lie)
T	F	passed, no bike	**F (the promise is broken!)**
T	T	passed, got the bike	T

The promise is only broken in one case: the condition happened and the result did not. When the condition did not happen, the promise was simply never "tested", so it cannot be false; in two-valued logic, "not false" means true.

Watch out — Implication is not causality

$p \to q$ only says "it is not the case that p is true and q false". It does not say that p *causes* q. "If $2 + 2 = 5$ then the Moon is made of cheese" is *true* (false antecedent). "If Rome is in Italy then $2 + 2 = 4$" is *true* too, although the two facts have nothing to do with each other. This is called **material implication**.

Why this matters in practice

Every `if` in a program, every rule in a firewall or a smart-home app is an implication. Smart-home rule: $motion \wedge night \to light_on$. During the day the rule is "not tested" and the light can be on or off without violating it, exactly as the truth table says. A *logic-based agent* (Section 8) is little more than a collection of such rules plus a reasoning engine.

2.2.3 Building complex formulas

[Slides · Propositional logic: 18]

Connectives can be nested as deeply as we want, and parentheses disambiguate. If p, q, r, s, t are atomic propositions, then these are formulae:

$$p \wedge q \to r \qquad p \wedge (q \to r) \qquad (p \wedge (q \to r)) \vee s \qquad ((p \wedge (q \to r)) \vee s) \wedge t$$

whereas none of these is a formula:

$$p \wedge \qquad p \wedge q) \qquad p\neg$$

($p\wedge$ misses the right operand; $p\wedge q)$ has an unbalanced parenthesis; in $p\neg$ the negation is on the wrong side).

Definition — Well-formed formulas of propositional logic

[Not in the slides: added material]

The set of formulas is defined *recursively*:

1. every atomic proposition $(p, q, r, \ldots)$ is a formula;
2. if φ is a formula, then $\neg\varphi$ is a formula;
3. if φ and ψ are formulas, then $(\varphi \wedge \psi)$, $(\varphi \vee \psi)$, $(\varphi \to \psi)$, $(\varphi \leftrightarrow \psi)$ are

formulas;
4. nothing else is a formula.

Key result — Precedence (how to read a formula without all its parentheses)

[Not in the slides: added material]

From the strongest (binds first) to the weakest: $\neg$, then $\wedge$, then $\vee$, then $\rightarrow$, then $\leftrightarrow$. So $p \wedge q \rightarrow r$ means $(p \wedge q) \rightarrow r$, and $\neg p \vee q$ means $(\neg p) \vee q$. Implication associates to the right: $p \rightarrow q \rightarrow r$ means $p \rightarrow (q \rightarrow r)$. When in doubt, write the parentheses.

Worked example — the structure (parse tree) of a formula

The formula $((p \wedge (q \rightarrow r)) \vee s) \wedge t$ from slide 18 has this tree: the **main connective** (the last one applied) is at the root.

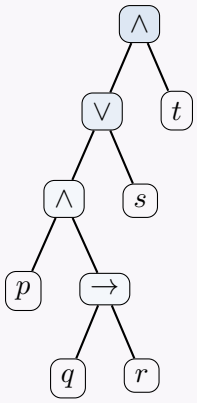


Figure 7: Parse tree: the leaves are atomic propositions, internal nodes are connectives. Truth values are computed bottom-up, from the leaves to the root.

2.3 Valuations, tautologies and consistency

[Slides · Propositional logic: 19–21]

2.3.1 Valuations: giving a truth value to a formula

[Slides · Propositional logic: 19–20]

Given a formula, can you tell if it is true or not? **No**: you usually need to know the truth values of its atomic propositions first. Is $p \vee q$ true? It depends on p and q.

Definition — Valuation (slide 19)

A **valuation** is a function v that assigns a truth value (T or F) to each atomic (primitive) proposition. In a programming language it would be a function `Val(p: AtomicProp): BOOLEAN`. Given a valuation, we can compute for *any* formula

whether it is true or false, by applying the truth tables bottom-up.

Worked example — evaluating $(p \vee q) \to r$ (slide 20)

Let $v(p) = \mathsf{F}$, $v(q) = \mathsf{T}$, $v(r) = \mathsf{F}$. Then

$$\begin{aligned} v\big((p \vee q) \to r\big) &= (v(p) \vee v(q)) \to v(r) && \triangleleft \text{ replace each letter by its value} \\ &= (\mathsf{F} \vee \mathsf{T}) \to \mathsf{F} \\ &= \mathsf{T} \to \mathsf{F} && \triangleleft\ \mathsf{F} \vee \mathsf{T} = \mathsf{T} \text{ (table of } \vee) \\ &= \mathsf{F} && \triangleleft\ \mathsf{T} \to \mathsf{F} = \mathsf{F} \text{ (table of } \to) \end{aligned}$$

So under this valuation the formula is **false**: the antecedent $p \vee q$ holds but the consequent r does not, which is the only way to falsify an implication.

Key result — Rows = valuations

Each row of a truth table corresponds to exactly one valuation of the atomic propositions of the formula. A truth table therefore shows the value of the formula under *all* possible valuations (2^n of them).

2.3.2 Tautologies, consistency, inconsistency

[Slides · Propositional logic: 21]

Some formulas have interesting properties that do not depend on the particular valuation.

Definition — Tautology, consistent, inconsistent (slide 21)

1. A formula is a **tautology** iff it is true under *every* valuation.
2. A formula is **consistent** iff it is true under *at least one* valuation (also called *satisfiable*).
3. A formula is **inconsistent** iff it is *not* made true under any valuation (also called *contradiction* or *unsatisfiable*).

Since each row of a truth table is a valuation, truth tables decide these properties: look at the last column. All T ⇒ tautology; at least one T ⇒ consistent; no T ⇒ inconsistent.

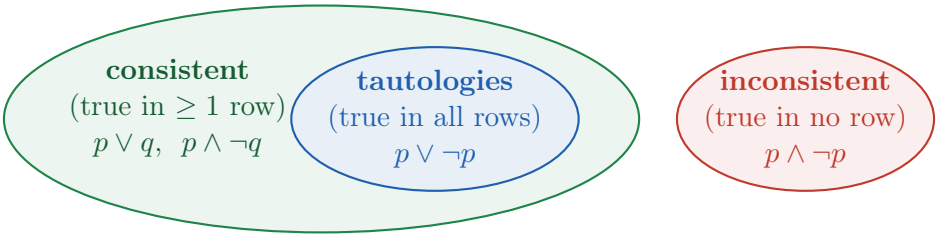


Figure 8: Every formula is either consistent or inconsistent. Tautologies are a special case of consistent formulas. The negation of a tautology is inconsistent and vice versa.

Worked example — one formula of each kind

p	$p \vee \neg p$	$p \wedge \neg p$	$p \to \neg p$
F	T	F	T
T	T	F	F
	tautology	inconsistent	consistent, not a tautology

$p \vee \neg p$ ("it rains or it does not rain") is always true: it carries no information about the world. $p \wedge \neg p$ can never be true.

Worked example — modus ponens is a tautology, affirming the consequent is not

[Not in the slides: added material]

Consider the two formulas $A = (p \wedge (p \to q)) \to q$ and $B = (q \wedge (p \to q)) \to p$. Full truth table, one column per sub-formula:

p	q	$p \to q$	$p \wedge (p \to q)$	A	$q \wedge (p \to q)$	B
F	F	T	F	T	F	T
F	T	T	F	T	T	F
T	F	F	F	T	F	T
T	T	T	T	T	T	T

A is a **tautology**: whenever p and $p \to q$ are both true, q is true. This is the reasoning pattern *modus ponens* (Section 2.4). B is **not**: in row 2 (p false, q true) both q and $p \to q$ are true but p is false. Concretely: "if it rains, the street is wet; the street is wet; so it rains" is wrong, because someone may have washed the street.

2.3.3 Logical equivalence and logical consequence

[Not in the slides: added material]

Two more notions, which the slides use implicitly (slide 16 and the proof-theory slides), are worth stating precisely.

Definition — Logical equivalence

Two formulas φ and ψ are **logically equivalent**, written $\varphi \equiv \psi$, iff they have the same truth value under *every* valuation, i.e. iff $\varphi \leftrightarrow \psi$ is a **tautology**.

Watch out — About slide 16

Slide 16 says: "If $p \leftrightarrow q$ is true, then p and q are said to be logically equivalent". Read it carefully: what is meant is that $p \leftrightarrow q$ is true under *every* valuation (a tautology). If $p \leftrightarrow q$ is true under *one* valuation, p and q merely happen to have the same value there. Example: $p =$ "it is Monday" and $q =$ "it rains" may both be true today, so $p \leftrightarrow q$ is true today, but they are certainly not logically equivalent.

Definition — Logical consequence (valid argument)

A formula ψ is a **logical consequence** of the premises $\varphi_1, \ldots, \varphi_n$, written $\varphi_1, \ldots, \varphi_n \models \psi$, iff *every valuation that makes all the premises true also makes* ψ *true*. Equivalently: there is no valuation with all premises true and ψ false; equivalently, $(\varphi_1 \wedge \cdots \wedge \varphi_n) \to \psi$ is a tautology. An argument whose conclusion is a logical consequence of its premises is called **(deductively) valid**.

Worked example — a few useful equivalences, checked by truth table

p	q	$p \to q$	$\neg p \vee q$	$\neg q \to \neg p$	$\neg(p \wedge q)$	$\neg p \vee \neg q$
F	F	T	T	T	T	T
F	T	T	T	T	T	T
T	F	F	F	F	T	T
T	T	T	T	T	F	F

Equal columns $\Rightarrow$ equivalent formulas: $p \to q \equiv \neg p \vee q \equiv \neg q \to \neg p$ (the last one is the *contrapositive*), and $\neg(p \wedge q) \equiv \neg p \vee \neg q$ (one of De Morgan's laws, used again in Section 2.4).

Why this matters in practice — Truth tables in real software

Deciding whether a formula is consistent ("is there a valuation that makes it true?") is the famous **SAT problem**. A truth table needs 2^n rows, which explodes quickly (the combinatorial explosion of Section 1.3); modern **SAT solvers** use clever search instead and routinely handle formulas with millions of variables. They are used to verify hardware circuits, find bugs in software, schedule exams and timetables, and solve puzzles such as Sudoku ("cell (1,1) contains 5" is an atomic proposition).

2.4 Proof theory: natural deduction

[Slides · Proof theory (prop.): 1–11]

Truth tables check validity by brute force: they look at *all* valuations. With 20 atomic propositions that is more than a million rows. Humans (and many programs) prove things differently: they start from what they know and take small, obviously correct steps until they reach the conclusion. This is what a **proof system** formalises.

2.4.1 What is a proof?

[Slides · Proof theory (prop.): 1]

Definition — Proof system, proof, derivation (page 1)

A **proof system** is a set of rules for constructing proofs. A **proof** is a series of sentences, each of which is either a **premise** or is justified by applying one of the

rules of the system to *earlier* sentences of the series. A sentence justified in this way is said to be **derived** from the premises.

The rules are designed with a very important limitation: they only allow you to derive a conclusion from premises if the argument is **deductively valid**, i.e. if it is not possible for all the premises to be true while the conclusion is false (the notion of logical consequence of Section 2.3). This is what makes it legitimate to call the series of sentences a proof.

The system used in the course is **natural deduction**: each rule lets you add a sentence that must be true given that certain earlier sentences are true, just like you would convince a friend "in clear, simple steps". For each connective there are two kinds of rules:

- **Elim** (elimination) rules break apart a sentence containing the connective (from $p \wedge q$ get p);
- **Intro** (introduction) rules build a sentence containing the connective (from p and q get $p \wedge q$);

plus a special rule **Reit** (reiteration: repeat a sentence you already have). These basic rules are enough to prove *any* truth-functionally valid argument. Since some proofs become very long, a few **derived rules** are added later to make proofs shorter (they prove nothing new).

Key result — How a proof is written (Fitch format)

Sentences are numbered consecutively. The premises come first, and a horizontal line is drawn below the last premise. Every following line is a (subsidiary or final) conclusion; next to it we write the rule that justifies it and the numbers of the earlier lines it is applied to, e.g. "$\wedge$ Elim: 1". Temporary assumptions open a **subproof**, drawn further to the right with its own vertical line (see below).

Intuition — Proofs and truth tables are two roads to the same place

A truth table asks: "in every possible world where the premises hold, does the conclusion hold?" A proof says: "here is a chain of small steps, each obviously safe, from the premises to the conclusion". For propositional logic the two roads lead to exactly the same valid arguments: the proof system is **sound** (it never proves an invalid argument) and **complete** (every valid argument has a proof).

2.4.2 Basic rules without subproofs

[Slides · Proof theory (prop.): 1–2]

Key result — Reit, $\wedge$ Elim, $\wedge$ Intro, $\rightarrow$ Elim, $\vee$ Intro

Rule	Schema	In words
Reit	$\dfrac{p}{p}$	you may repeat any sentence you already have
$\wedge$ Elim	$\dfrac{p \wedge q}{p} \quad \dfrac{p \wedge q}{q}$	from a conjunction, write either conjunct (as many times as you like)
$\wedge$ Intro	$\dfrac{p \quad q}{p \wedge q}$	join any two sentences you already have with $\wedge$
$\rightarrow$ Elim	$\dfrac{p \rightarrow q \quad p}{q}$	from a conditional and its antecedent, write its consequent (**Modus Ponens**)
$\vee$ Intro	$\dfrac{p}{p \vee q} \quad \dfrac{p}{q \vee p}$	from any sentence, write a disjunction containing it, with *any* other sentence, in either order

Read a schema as "if you have the sentences above the line, you may write the one below".

Why are they safe? $\wedge$ Elim: if "p and q" is true, p is true. $\wedge$ Intro: if both are true, so is their conjunction. $\rightarrow$ Elim: the only way $p \rightarrow q$ can be true with p true is that q is true (look at the truth table). $\vee$ Intro: a disjunction only needs *one* true disjunct, so adding any other disjunct is harmless.

Worked example — Example 1: $\{b \wedge c,\ j \wedge k\} \mathrel{/\!\therefore} c \wedge k$

1	$b \wedge c$	Premise
2	$j \wedge k$	Premise
3	c	$\wedge$ Elim: 1
4	k	$\wedge$ Elim: 2
5	$c \wedge k$	$\wedge$ Intro: 3, 4

(Premises are listed in braces; "$/\,\therefore$" reads "therefore".)

Worked example — Example 2: $\{g,\ h,\ (g \wedge h) \to (r \wedge s)\} \mathrel{/} \therefore\ s$

1	g	Premise
2	h	Premise
3	$(g \wedge h) \to (r \wedge s)$	Premise
4	$g \wedge h$	$\wedge$ Intro: 1, 2
5	$r \wedge s$	$\to$ Elim: 3, 4
6	s	$\wedge$ Elim: 5

Strategy: the conclusion s is "inside" $r \wedge s$, which is the consequent of line 3. To unlock it with $\to$ Elim we need the antecedent $g \wedge h$, which we build first.

Watch out — Modus Ponens vs. Affirming the Consequent

$\to$ Elim is the valid argument form **Modus Ponens**: $p \to q,\ p \mathrel{/} \therefore\ q$. A common error is the similar but **invalid** pattern **Affirming the Consequent**: $p \to q,\ q \mathrel{/} \therefore\ p$. The truth table of Section 2.3 gives the counterexample (p false, q true). "If the battery is dead, the car won't start; the car won't start; so the battery is dead" is wrong: maybe the tank is empty.

Worked example — Example 3: $\{(a \vee b) \to c,\ b\} \mathrel{/} \therefore\ c$

1	$(a \vee b) \to c$	Premise
2	b	Premise
3	$a \vee b$	$\vee$ Intro: 2
4	c	$\to$ Elim: 1, 3

From b, $\vee$ Intro could build $a \vee b$ or $b \vee a$ (or $b \vee z$ for any z). We choose $a \vee b$ because that is exactly the antecedent we need for line 1.

Exercise G1.A — From English to a proof (page 2–3)

Identify the premises and conclusion, translate using the letters given, then build a proof.

1. The starter must be malfunctioning. The car won't start, but the lights are working. If the lights work, the battery must be charged. If the car won't start when the battery is charged, there must be a problem with the starter. (s = the starter is functioning properly; c = the car starts; w = the lights are working; b = the battery is charged.)
2. If Mia moved out of her parents' house, she must be able to afford to. She can afford it only if she found both a roommate and a job. Since she moved out, she must have found a job. (m = Mia moved out; a = Mia can afford to move out; r = Mia found a roommate; j = Mia found a job.)

Solution

1. The conclusion is the first sentence ("the starter must be malfunctioning" $= \neg s$); "but" is logically an "and". Premises: $\neg c \land w$, $w \to b$, $(\neg c \land b) \to \neg s$.

1	$\neg c \land w$	Premise
2	$w \to b$	Premise
3	$(\neg c \land b) \to \neg s$	Premise
4	$\neg c$	$\land$ Elim: 1
5	w	$\land$ Elim: 1
6	b	$\to$ Elim: 2, 5
7	$\neg c \land b$	$\land$ Intro: 4, 6
8	$\neg s$	$\to$ Elim: 3, 7

2. "A only if B" means $A \to B$ (if A happens, B must hold). Premises: $m \to a$, $a \to (r \land j)$, m. Conclusion: j.

1	$m \to a$	Premise
2	$a \to (r \land j)$	Premise
3	m	Premise
4	a	$\to$ Elim: 1, 3
5	$r \land j$	$\to$ Elim: 2, 4
6	j	$\land$ Elim: 5

Lesson: "A only if B" is $A \to B$, *not* $B \to A$. "If" introduces the antecedent, "only if" introduces the consequent.

Exercise G1.B — Derive the conclusion from the premises (page 3)

1. $a,\ a \to b,\ (a \land b) \to c \ /\ \therefore\ b \land c$
2. $g \land h,\ g \to (d \to k),\ h \to d \ /\ \therefore\ d \land k$
3. $a \land b,\ (b \lor c) \to (d \land e),\ e \to (b \to s) \ /\ \therefore\ s$
4. $(m \lor o) \to k,\ k \to (l \land r),\ m \ /\ \therefore\ k \land (r \lor s)$

Solution

1.

1	a	Premise
2	$a \to b$	Premise
3	$(a \wedge b) \to c$	Premise
4	b	$\to$ Elim: 2, 1
5	$a \wedge b$	$\wedge$ Intro: 1, 4
6	c	$\to$ Elim: 3, 5
7	$b \wedge c$	$\wedge$ Intro: 4, 6

2.

1	$g \wedge h$	Premise
2	$g \to (d \to k)$	Premise
3	$h \to d$	Premise
4	g	$\wedge$ Elim: 1
5	h	$\wedge$ Elim: 1
6	$d \to k$	$\to$ Elim: 2, 4
7	d	$\to$ Elim: 3, 5
8	k	$\to$ Elim: 6, 7
9	$d \wedge k$	$\wedge$ Intro: 7, 8

3.

1	$a \wedge b$	Premise
2	$(b \vee c) \to (d \wedge e)$	Premise
3	$e \to (b \to s)$	Premise
4	b	$\wedge$ Elim: 1
5	$b \vee c$	$\vee$ Intro: 4
6	$d \wedge e$	$\to$ Elim: 2, 5
7	e	$\wedge$ Elim: 6
8	$b \to s$	$\to$ Elim: 3, 7
9	s	$\to$ Elim: 8, 4

4.

1	$(m \vee o) \to k$	Premise
2	$k \to (l \wedge r)$	Premise
3	m	Premise
4	$m \vee o$	$\vee$ Intro: 3
5	k	$\to$ Elim: 1, 4
6	$l \wedge r$	$\to$ Elim: 2, 5
7	r	$\wedge$ Elim: 6
8	$r \vee s$	$\vee$ Intro: 7
9	$k \wedge (r \vee s)$	$\wedge$ Intro: 5, 8

Lesson: work *backwards* from the conclusion. In 4, $k \wedge (r \vee s)$ needs k and $r \vee s$; $r \vee s$ needs only r ($\vee$ Intro); r is inside $l \wedge r$; and so on, until you reach the premises.

2.4.3 Rules with subproofs

[Slides · Proof theory (prop.): 3–5]

The remaining Intro and Elim rules need a new ingredient: we temporarily **assume** something that was not given as a premise, see what follows, and then give the assumption up. The part of a proof in which we hold a temporary assumption is a **subproof**, drawn to the right of a new vertical line (its first line, the assumption, is underlined).

Key result — The subproof restriction (page 10)

A rule may only be applied to sentences that appear (1) **earlier** in the proof and (2) **outside any subproof that has already ended**. Once a subproof ends, its lines may depend on an assumption we no longer hold, so they cannot be used individually any more; the only way to use a finished subproof is as a whole, with

one of the four rules below. Subproofs can be nested, but an inner subproof must end before the outer one.

Key result — ¬ Intro, ¬ Elim, → Intro, ∨ Elim

1	|	p	Assumption
2	|	⋮	
3	|	q	
4	|	$\neg q$	
5	$\neg p$		¬ Intro

¬ Intro (proof by contradiction)

1	|	$\neg p$	Assumption
2	|	⋮	
3	|	q	
4	|	$\neg q$	
5	p		¬ Elim

¬ Elim

1	|	p	Assumption
2	|	⋮	
3	|	q	
4	$p \to q$		→ Intro

→ Intro (conditional proof)

1	$p \vee q$		
2	|	p	Assumption
3	|	r	
4	|	q	Assumption
5	|	r	
6	r		∨ Elim

∨ Elim (proof by cases)

- ¬ **Intro**: if an assumption leads to a sentence *and* its negation (a contradiction), the assumption cannot be true: close the subproof and write its negation.
- ¬ **Elim**: exactly like ¬ Intro, but the assumption is a negation $\neg p$; from the contradiction, conclude p.
- → **Intro**: assume the antecedent p; if you derive q, close the subproof and write $p \to q$.
- ∨ **Elim**: from $p \vee q$, open two subproofs, one assuming p and one assuming q; anything r derived in *both* can be written outside them.

Citation style: the assumption line through the contradictory pair, e.g. "¬ Intro: 3–(4,5)"; for ∨ Elim, the disjunction and both subproofs, e.g. "∨ Elim: 1, 4–6, 7–9".

Intuition — Everyday versions of these rules

- ¬ *Intro, reductio ad absurdum*: "Suppose the butler did it. Then he was in the kitchen at 9. But he was seen in the garden at 9. Contradiction: the butler did not do it."
- → *Intro, conditional proof*: "Suppose I take the 8:00 train. Then I arrive at 9:30, so I am in time for the meeting. Hence: if I take the 8:00 train, I am in time." You never claimed to take the train.
- ∨ *Elim, proof by cases*: "Either it rains or it is sunny. If it rains I take the bus; if it is sunny I take the bus (too hot to walk). So I take the bus."

Worked example — Example 4 ($\neg$ Intro): $\{\neg(g \wedge h),\ h\} \therefore \neg g$

"It's not the case that both Gary and Harvey will be promoted. Harvey is getting a promotion. So Gary won't be promoted."

1	$\neg(g \wedge h)$	Premise
2	h	Premise
3	$\quad g$	Assumption
4	$\quad g \wedge h$	$\wedge$ Intro: 2, 3
5	$\quad \neg(g \wedge h)$	Reit: 1
6	$\neg g$	$\neg$ Intro: 3—(4, 5)

Suppose Gary will be promoted (3). Then both will be promoted (4). But we were told they will not both be promoted (5). So the assumption is false (6). Note the use of Reit: the rule requires the contradictory pair *inside* the subproof, so we copy line 1 in.

Watch out — Wording in the slides

In the slides the assumption lines of subproofs are labelled "Premise". They are *temporary* premises (assumptions) that are discharged when the subproof ends, unlike the real premises at the top. In these notes they are labelled "Assumption".

Worked example — Example 5 ($\neg$ Elim): $\{\neg a \to n,\ \neg(n \vee e)\} \therefore a$

"If Alicia doesn't get the job, Nina will. But there is no way Nina or Eloise will get the job. So Alicia will get it."

1	$\neg a \to n$	Premise
2	$\neg(n \vee e)$	Premise
3	$\quad \neg a$	Assumption
4	$\quad n$	$\to$ Elim: 1, 3
5	$\quad n \vee e$	$\vee$ Intro: 4
6	$\quad \neg(n \vee e)$	Reit: 2
7	a	$\neg$ Elim: 3—(5, 6)

Worked example — Example 6 ($\to$ Intro): $\{a \to p,\ p \to (m \wedge r),\ m \to q\} \therefore a \to q$

"If Alberto gets his way, the policy will be approved. In that case, Marta and Roger will be angry. Marta will quit if she is angry. So if Alberto gets his way, Marta will quit."

1	$a \to p$	Premise
2	$p \to (m \wedge r)$	Premise
3	$m \to q$	Premise
4	$\quad a$	Assumption
5	$\quad p$	$\to$ Elim: 1, 4
6	$\quad m \wedge r$	$\to$ Elim: 2, 5
7	$\quad m$	$\wedge$ Elim: 6
8	$\quad q$	$\to$ Elim: 3, 7
9	$a \to q$	$\to$ Intro: 4—8

Worked example — Example 7 (nested subproofs): $\{b \to \neg(c \wedge d),\ a \to (b \to d)\} \mathrel{/\!\!\therefore} (a \wedge b) \to \neg c$

1	$b \to \neg(c \wedge d)$	Premise
2	$a \to (b \to d)$	Premise
3	$\quad a \wedge b$	Assumption
4	$\quad a$	$\wedge$ Elim: 3
5	$\quad b \to d$	$\to$ Elim: 2, 4
6	$\quad b$	$\wedge$ Elim: 3
7	$\quad d$	$\to$ Elim: 5, 6
8	$\quad\quad c$	Assumption
9	$\quad\quad c \wedge d$	$\wedge$ Intro: 7, 8
10	$\quad\quad \neg(c \wedge d)$	$\to$ Elim: 1, 6
11	$\quad \neg c$	$\neg$ Intro: 8—(9, 10)
12	$(a \wedge b) \to \neg c$	$\to$ Intro: 3—11

To prove an implication, assume its antecedent (3) and aim for the consequent $\neg c$; to prove a negation, assume the un-negated sentence (8) and aim for a contradiction. Inside the inner subproof we may use lines 1–7, because they are earlier and in subproofs that are still open.

Worked example — Example 8 ($\vee$ Elim): Rosa arrives late

"Rosa will get a ride with either Don or Glenn. If Don gives her a ride, she'll be picked up late and she'll arrive late. If Glenn gives her a ride, he'll make a stop along the way and Rosa will arrive late. So Rosa will arrive late."

1	$d \lor g$	Premise
2	$d \to (p \land a)$	Premise
3	$g \to (s \land a)$	Premise
4	$\quad d$	Assumption
5	$\quad p \land a$	$\to$ Elim: 2, 4
6	$\quad a$	$\land$ Elim: 5
7	$\quad g$	Assumption
8	$\quad s \land a$	$\to$ Elim: 3, 7
9	$\quad a$	$\land$ Elim: 8
10	a	$\lor$ Elim: 1, 4—6, 7—9

Since Rosa will be late either way, we don't need to know *which* alternative of line 1 is true. (The slides contain a typo, "If Don gives May a ride": it is Rosa.)

Worked example — Example 9 ($\lor$ Elim): $\{l \to (h \land u),\ i \to s,\ l \lor i\} \mathrel{/\!\!\therefore} u \lor s$

"If Luis is given the job, he'll be happy but Igor will be upset. If the job goes to Igor, Luis will be upset. One of them will get the job, so one of them will be upset."

1	$l \to (h \land u)$	Premise
2	$i \to s$	Premise
3	$l \lor i$	Premise
4	$\quad l$	Assumption
5	$\quad h \land u$	$\to$ Elim: 1, 4
6	$\quad u$	$\land$ Elim: 5
7	$\quad u \lor s$	$\lor$ Intro: 6
8	$\quad i$	Assumption
9	$\quad s$	$\to$ Elim: 2, 8
10	$\quad u \lor s$	$\lor$ Intro: 9
11	$u \lor s$	$\lor$ Elim: 3, 4—7, 8—10

The two cases end with *different* facts (u, s), so we first make both cases end with the *same* sentence $u \lor s$ using $\lor$ Intro.

Exercise G2 — Derive the conclusion from the premises, basic rules only (page 6)

1. $\neg b,\ \neg(\neg a \land \neg b) \mathbin{/} \therefore\ a$
2. $j \to (k \land m),\ m \to \neg k \mathbin{/} \therefore\ \neg j$
3. $c \to (d \land e),\ d \to \neg h \mathbin{/} \therefore\ c \to \neg h$
4. $(m \lor r) \to s,\ s \to (p \to q) \mathbin{/} \therefore\ (r \land p) \to q$
5. $(b \land c) \to d,\ (d \land g) \to h,\ b \land g \mathbin{/} \therefore\ c \to h$
6. $r \to \neg(p \land q),\ w \to (r \to q) \mathbin{/} \therefore\ w \to \neg(p \land r)$
7. $a \lor n,\ n \to (e \land s),\ a \to s \mathbin{/} \therefore\ s$
8. $j \to (k \lor l),\ (j \land k) \to m,\ j,\ l \to (m \land o) \mathbin{/} \therefore\ m$
9. $a \to (b \lor c),\ d \to \neg b,\ c \to \neg(a \land d) \mathbin{/} \therefore\ a \to \neg d$

Solution

All nine arguments are valid (checked by truth table). Strategy hints: to prove $\neg X$, assume X and find a contradiction ($\neg$ Intro); to prove $X \to Y$, assume X and derive Y ($\to$ Intro); to use $X \lor Y$, reason by cases ($\lor$ Elim).

1.

1	$\neg b$	Premise
2	$\neg(\neg a \land \neg b)$	Premise
3	$\quad \neg a$	Assumption
4	$\quad \neg a \land \neg b$	$\land$ Intro: 3, 1
5	$\quad \neg(\neg a \land \neg b)$	Reit: 2
6	a	$\neg$ Elim: 3—(4, 5)

2.

1	$j \to (k \land m)$	Premise
2	$m \to \neg k$	Premise
3	$\quad j$	Assumption
4	$\quad k \land m$	$\to$ Elim: 1, 3
5	$\quad k$	$\land$ Elim: 4
6	$\quad m$	$\land$ Elim: 4
7	$\quad \neg k$	$\to$ Elim: 2, 6
8	$\neg j$	$\neg$ Intro: 3—(5, 7)

3.

1	$c \to (d \land e)$	Premise
2	$d \to \neg h$	Premise
3	$\quad c$	Assumption
4	$\quad d \land e$	$\to$ Elim: 1, 3
5	$\quad d$	$\land$ Elim: 4
6	$\quad \neg h$	$\to$ Elim: 2, 5
7	$c \to \neg h$	$\to$ Intro: 3—6

4.

1	$(m \lor r) \to s$	Premise
2	$s \to (p \to q)$	Premise
3	$\quad r \land p$	Assumption
4	$\quad r$	$\land$ Elim: 3
5	$\quad m \lor r$	$\lor$ Intro: 4
6	$\quad s$	$\to$ Elim: 1, 5
7	$\quad p \to q$	$\to$ Elim: 2, 6
8	$\quad p$	$\land$ Elim: 3
9	$\quad q$	$\to$ Elim: 7, 8
10	$(r \land p) \to q$	$\to$ Intro: 3—9

5.

1	$(b \land c) \to d$	Premise
2	$(d \land g) \to h$	Premise
3	$b \land g$	Premise
4	$\quad c$	Assumption
5	$\quad b$	$\land$ Elim: 3
6	$\quad b \land c$	$\land$ Intro: 5, 4
7	$\quad d$	$\to$ Elim: 1, 6
8	$\quad g$	$\land$ Elim: 3
9	$\quad d \land g$	$\land$ Intro: 7, 8
10	$\quad h$	$\to$ Elim: 2, 9
11	$c \to h$	$\to$ Intro: 4—10

6. Two nested subproofs: $\to$ Intro outside, $\neg$ Intro inside.

1	$r \to \neg(p \wedge q)$	Premise
2	$w \to (r \to q)$	Premise
3	$\quad w$	Assumption
4	$\quad r \to q$	$\to$ Elim: 2, 3
5	$\quad\quad p \wedge r$	Assumption
6	$\quad\quad r$	$\wedge$ Elim: 5
7	$\quad\quad q$	$\to$ Elim: 4, 6
8	$\quad\quad p$	$\wedge$ Elim: 5
9	$\quad\quad p \wedge q$	$\wedge$ Intro: 8, 7
10	$\quad\quad \neg(p \wedge q)$	$\to$ Elim: 1, 6
11	$\quad \neg(p \wedge r)$	$\neg$ Intro: 5—(9, 10)
12	$w \to \neg(p \wedge r)$	$\to$ Intro: 3—11

7.

1	$a \vee n$	Premise
2	$n \to (e \wedge s)$	Premise
3	$a \to s$	Premise
4	$\quad a$	Assumption
5	$\quad s$	$\to$ Elim: 3, 4
6	$\quad n$	Assumption
7	$\quad e \wedge s$	$\to$ Elim: 2, 6
8	$\quad s$	$\wedge$ Elim: 7
9	s	$\vee$ Elim: 1, 4—5, 6—8

8.

1	$j \to (k \vee l)$	Premise
2	$(j \wedge k) \to m$	Premise
3	j	Premise
4	$l \to (m \wedge o)$	Premise
5	$k \vee l$	$\to$ Elim: 1, 3
6	$\quad k$	Assumption
7	$\quad j \wedge k$	$\wedge$ Intro: 3, 6
8	$\quad m$	$\to$ Elim: 2, 7
9	$\quad l$	Assumption
10	$\quad m \wedge o$	$\to$ Elim: 4, 9
11	$\quad m$	$\wedge$ Elim: 10
12	m	$\vee$ Elim: 5, 6—8, 9—11

9. The hardest one. Plan: assume a, then assume d and look for a contradiction between $a \wedge d$ and $\neg(a \wedge d)$. To get $\neg(a \wedge d)$ we need c, which must come from $b \vee c$ by cases. In the case b we have both b and $\neg b$, and from a contradiction anything follows (here c, via $\neg$ Elim).

1	$a \to (b \vee c)$	Premise
2	$d \to \neg b$	Premise
3	$c \to \neg(a \wedge d)$	Premise
4	$\quad a$	Assumption
5	$\quad b \vee c$	$\to$ Elim: 1, 4
6	$\quad\quad d$	Assumption
7	$\quad\quad \neg b$	$\to$ Elim: 2, 6
8	$\quad\quad a \wedge d$	$\wedge$ Intro: 4, 6
9	$\quad\quad\quad b$	Assumption
10	$\quad\quad\quad\quad \neg c$	Assumption
11	$\quad\quad\quad\quad b$	Reit: 9
12	$\quad\quad\quad\quad \neg b$	Reit: 7
13	$\quad\quad\quad c$	$\neg$ Elim: 10—(11, 12)
14	$\quad\quad\quad c$	Assumption
15	$\quad\quad\quad c$	Reit: 14
16	$\quad\quad c$	$\vee$ Elim: 5, 9—13, 14—15
17	$\quad\quad \neg(a \wedge d)$	$\to$ Elim: 3, 16
18	$\quad \neg d$	$\neg$ Intro: 6—(8, 17)
19	$a \to \neg d$	$\to$ Intro: 4—18

Lesson: lines 9–16 prove the *disjunctive syllogism* ($b \vee c,\ \neg b \ /\ \therefore\ c$) with basic rules only. It is so useful that it becomes a derived rule in the next sub-section, and with it this proof shrinks from 19 to 11 lines.

2.4.4 Derived rules

[Slides · Proof theory (prop.): 6–8, 11]

Derived rules express patterns of argument that we could prove using only the basic rules. Anything we prove with them, we could prove without them: they are useful not because of *what* they let us prove, but because they let us prove it more quickly.

Key result — The derived rules

Rule	Schema	In words
Modus Tollens	$\dfrac{p \to q \qquad \neg q}{\neg p}$	from a conditional and the negation of its consequent, derive the negation of its antecedent
Hypothetical syllogism	$\dfrac{p \to q \qquad q \to r}{p \to r}$	chain conditionals (any number, in any order)
Disjunctive syllogism	$\dfrac{p \lor q \quad \neg p}{q} \qquad \dfrac{p \lor q \quad \neg q}{p}$	from a disjunction and the negation of one disjunct, derive the other
Double negation ($\neg\neg$)	$\neg\neg p \equiv p$	drop or add $\neg\neg$, also inside a longer sentence
De Morgan's laws	$\neg(p \land q) \equiv \neg p \lor \neg q$ $\neg(p \lor q) \equiv \neg p \land \neg q$	substitute one form for the other, also inside a longer sentence

In proofs these rules are abbreviated **MT** (Modus Tollens), **HS** (hypothetical syllogism), **DS** (disjunctive syllogism), $\neg\neg$ and **DeM** (De Morgan). Double negation and De Morgan are **substitution rules**: since the two forms are logically equivalent (same truth table column), one may replace the other even as a *part* of a bigger sentence. The other rules apply only to whole lines.

Worked example — why Modus Tollens is safe: a proof with basic rules

[Not in the slides: added material]

1	$p \to q$	Premise
2	$\neg q$	Premise
3	$\quad p$	Assumption
4	$\quad q$	$\to$ Elim: 1, 3
5	$\quad \neg q$	Reit: 2
6	$\neg p$	$\neg$ Intro: 3—(4, 5)

Every use of Modus Tollens can be replaced by these 4 extra lines: that is what "derived" means.

Worked example — Example 10 (Modus Tollens): Raul's newspaper

"Raul would have picked up his newspaper if he were home. He must not be home, because he did not pick up his paper." $\{h \to n,\ \neg n\} \mathrel{/} \therefore\ \neg h$

1	$h \to n$	Premise
2	$\neg n$	Premise
3	$\neg h$	MT: 1, 2

This may remind you of the invalid form *affirming the consequent*; the negations make the difference: Modus Tollens is valid.

Worked example — Examples 11–13 (hypothetical and disjunctive syllogism)

Ex. 11

1	$a \to b$	Premise
2	$b \to c$	Premise
3	$c \to d$	Premise
4	$d \to e$	Premise
5	$a \to e$	HS: 1—4

Ex. 12

1	$(j \wedge k) \to m$	Premise
2	$\neg l$	Premise
3	$k \vee l$	Premise
4	$\quad j$	Assumption
5	$\quad k$	DS: 2, 3
6	$\quad j \wedge k$	$\wedge$ Intro: 4, 5
7	$\quad m$	$\to$ Elim: 1, 6
8	$j \to m$	$\to$ Intro: 4—7

Ex. 13

1	$(p \wedge q) \vee (r \wedge s)$	Premise
2	$p \to \neg(q \vee u)$	Premise
3	$\quad p \wedge q$	Assumption
4	$\quad p$	$\wedge$ Elim: 3
5	$\quad q$	$\wedge$ Elim: 3
6	$\quad q \vee u$	$\vee$ Intro: 5
7	$\quad \neg(q \vee u)$	$\to$ Elim: 2, 4
8	$\neg(p \wedge q)$	$\neg$ Intro: 3—(6, 7)
9	$r \wedge s$	DS: 1, 8
10	s	$\wedge$ Elim: 9

In Example 12 the slides list j as a fourth premise, but it is really the assumption of the subproof (it is discharged by $\to$ Intro in line 8).

Worked example — Example 14 (double negation): Ned is sick

"If Ned wasn't sick or on vacation, he'd be at work. He's not at work and not on vacation, so he must be sick." $\{\neg(s \vee o) \to w,\ \neg w \wedge \neg o\} \mathrel{/} \therefore\ s$

1	$\neg(s \vee o) \to w$	Premise
2	$\neg w \wedge \neg o$	Premise
3	$\neg w$	$\wedge$ Elim: 2
4	$\neg\neg(s \vee o)$	MT: 1, 3
5	$s \vee o$	$\neg\neg$: 4
6	$\neg o$	$\wedge$ Elim: 2
7	s	DS: 5, 6

Modus Tollens negates the antecedent $\neg(s \vee o)$, producing $\neg\neg(s \vee o)$; double negation removes the two $\neg$.

Worked example — Example 15 (De Morgan): Javier is sick

"If Javier wasn't sick or on vacation, he'd be at work. He's neither at work nor on vacation, so he must be sick." $\{\neg(s \vee o) \to w,\ \neg(w \vee o)\} \mathrel{/} \therefore\ s$

1	$\neg(s \vee o) \to w$	Premise
2	$\neg(w \vee o)$	Premise
3	$\neg w \wedge \neg o$	DeM: 2
4	$\neg w$	$\wedge$ Elim: 3
5	$\neg\neg(s \vee o)$	MT: 1, 4
6	$s \vee o$	$\neg\neg$: 5
7	$\neg o$	$\wedge$ Elim: 3
8	s	DS: 6, 7

Watch out — Typos in Examples 14–16 of the slides

In Example 15 the slides justify lines 4 and 7 with "$\wedge$ Elim: 2", but line 2 is $\neg(w \vee o)$, not a conjunction: the conjunction is line 3 (obtained by De Morgan), so the correct citation is "$\wedge$ Elim: 3" (as written above). In Example 14 the premise set is printed as $\{\neg(s \vee o) \supset w,\ \neg w\}$, while the proof uses $\neg w \wedge \neg o$ (and $\supset$ is just another symbol for $\to$). In Example 16, "Modus Tollens: 4.5" means lines 4,5.

Worked example — Examples 16–17 (De Morgan inside longer proofs)

Ex. 16: "Ed couldn't have passed history. If he had passed history or sociology, he would have graduated. If he'd graduated, he would have gone to the ceremony, but he didn't." $\{(h \vee s) \to g,\ (g \to c) \wedge \neg c\} \mathrel{/} \therefore\ \neg h$

1	$(h \lor s) \to g$	Premise
2	$(g \to c) \land \neg c$	Premise
3	$g \to c$	$\land$ Elim: 2
4	$(h \lor s) \to c$	HS: 1, 3
5	$\neg c$	$\land$ Elim: 2
6	$\neg(h \lor s)$	MT: 4, 5
7	$\neg h \land \neg s$	DeM: 6
8	$\neg h$	$\land$ Elim: 7

Ex. 17

1	$a \to (b \land c)$	Premise
2	$c \to d$	Premise
3	$\quad \neg d$	Assumption
4	$\quad \neg c$	MT: 2, 3
5	$\quad \neg b \lor \neg c$	$\lor$ Intro: 4
6	$\quad \neg(b \land c)$	DeM: 5
7	$\quad \neg a$	MT: 1, 6
8	$\neg d \to \neg a$	$\to$ Intro: 3—7

Exercise G3 — Derive the conclusion, derived rules allowed (page 9)

1. $(a \land b) \to c,\ \neg c \land \neg d \ /\ \therefore\ \neg a \lor \neg b$
2. $\neg(j \land k) \to m,\ j,\ \neg(l \lor m) \ /\ \therefore\ \neg k$
3. $d \to c,\ c \to \neg g,\ g \lor h \ /\ \therefore\ d \to h$
4. $j \to k,\ \neg l \to \neg(k \lor m) \ /\ \therefore\ j \to l$
5. $\neg b \to \neg c,\ \neg(a \lor b),\ (\neg a \land \neg c) \to d \ /\ \therefore\ d$
6. $\neg g \to (h \lor e),\ \neg h \land \neg e,\ g \lor m \ /\ \therefore\ m$
7. $\neg(a \lor b) \to c,\ \neg(b \lor c) \ /\ \therefore\ \neg a$
8. $r \to s,\ g \to h,\ (s \lor u) \to g \ /\ \therefore\ r \to h$
9. $o \to j,\ j \to (k \lor m),\ \neg(j \land m) \ /\ \therefore\ o \to k$

Solution

Watch out: exercises 2, 6 and 7 are not valid as printed in the slides. A truth table check finds, for each of them, a valuation that makes all premises true and the conclusion false, so *no* correct proof can exist:

- **2**: $j = k = \mathsf{T}$, $l = m = \mathsf{F}$. The premises actually prove k (not $\neg k$): the conclusion was probably meant to be k.
- **6**: $g = \mathsf{T}$, $h = e = m = \mathsf{F}$. The premises prove g, and $g \lor m$ then says nothing about m. With the third premise $\neg g \lor m$ ("$g \to m$") the argument becomes valid.
- **7**: $a = \mathsf{T}$, $b = c = \mathsf{F}$. The premises prove a (not $\neg a$).

Below, the valid versions are proved.

1.

1	$(a \land b) \to c$	Premise
2	$\neg c \land \neg d$	Premise
3	$\neg c$	$\land$ Elim: 2
4	$\neg(a \land b)$	MT: 1, 3
5	$\neg a \lor \neg b$	DeM: 4

2 (conclusion corrected to k).

1	$\neg(j \land k) \to m$	Premise
2	j	Premise
3	$\neg(l \lor m)$	Premise
4	$\neg l \land \neg m$	DeM: 3
5	$\neg m$	$\land$ Elim: 4
6	$\neg\neg(j \land k)$	MT: 1, 5
7	$j \land k$	$\neg\neg$: 6
8	k	$\land$ Elim: 7

3.

1	$d \to c$	Premise
2	$c \to \neg g$	Premise
3	$g \lor h$	Premise
4	$\quad d$	Assumption
5	$\quad d \to \neg g$	HS: 1, 2
6	$\quad \neg g$	$\to$ Elim: 5, 4
7	$\quad h$	DS: 3, 6
8	$d \to h$	$\to$ Intro: 4—7

4.

1	$j \to k$	Premise
2	$\neg l \to \neg(k \lor m)$	Premise
3	$\quad j$	Assumption
4	$\quad k$	$\to$ Elim: 1, 3
5	$\quad k \lor m$	$\lor$ Intro: 4
6	$\quad \neg\neg(k \lor m)$	$\neg\neg$: 5
7	$\quad \neg\neg l$	MT: 2, 6
8	$\quad l$	$\neg\neg$: 7
9	$j \to l$	$\to$ Intro: 3—8

5.

1	$\neg b \to \neg c$	Premise
2	$\neg(a \lor b)$	Premise
3	$(\neg a \land \neg c) \to d$	Premise
4	$\neg a \land \neg b$	DeM: 2
5	$\neg a$	$\land$ Elim: 4
6	$\neg b$	$\land$ Elim: 4
7	$\neg c$	$\to$ Elim: 1, 6
8	$\neg a \land \neg c$	$\land$ Intro: 5, 7
9	d	$\to$ Elim: 3, 8

6 (third premise corrected to $\neg g \lor m$).

1	$\neg g \to (h \lor e)$	Premise
2	$\neg h \land \neg e$	Premise
3	$\neg g \lor m$	Premise
4	$\neg(h \lor e)$	DeM: 2
5	$\neg\neg g$	MT: 1, 4
6	m	DS: 3, 5

7 (conclusion corrected to a).

1	$\neg(a \lor b) \to c$	Premise
2	$\neg(b \lor c)$	Premise
3	$\neg b \land \neg c$	DeM: 2
4	$\neg c$	$\land$ Elim: 3
5	$\neg\neg(a \lor b)$	MT: 1, 4
6	$a \lor b$	$\neg\neg$: 5
7	$\neg b$	$\land$ Elim: 3
8	a	DS: 6, 7

8.

1	$r \to s$	Premise
2	$g \to h$	Premise
3	$(s \lor u) \to g$	Premise
4	$\quad r$	Assumption
5	$\quad s$	$\to$ Elim: 1, 4
6	$\quad s \lor u$	$\lor$ Intro: 5
7	$\quad g$	$\to$ Elim: 3, 6
8	$\quad h$	$\to$ Elim: 2, 7
9	$r \to h$	$\to$ Intro: 4—8

9.

1	$o \to j$	Premise
2	$j \to (k \lor m)$	Premise
3	$\neg(j \land m)$	Premise
4	$\quad o$	Assumption
5	$\quad j$	$\to$ Elim: 1, 4
6	$\quad k \lor m$	$\to$ Elim: 2, 5
7	$\quad \neg j \lor \neg m$	DeM: 3
8	$\quad \neg\neg j$	$\neg\neg$: 5
9	$\quad \neg m$	DS: 7, 8
10	$\quad k$	DS: 6, 9
11	$o \to k$	$\to$ Intro: 4—10

In exercise 6 (line 5), Modus Tollens gives $\neg\neg g$, which is exactly the negation of the first disjunct $\neg g$ of line 3, so disjunctive syllogism applies directly. **Lesson**: before trying to prove something, a quick truth-table check (or a counterexample) can save you a lot of time; proof rules can only prove valid arguments.

Cheat sheet — The proof rules at a glance (pages 10–11)

General restriction	a rule applies only to sentences that are earlier and outside any subproof that has already ended
Reit	repeat any sentence you already have
$\wedge$ Elim / $\wedge$ Intro	take a conjunct / join two sentences with $\wedge$
$\rightarrow$ Elim (Modus Ponens)	from $p \rightarrow q$ and p, write q
$\vee$ Intro	from p, write $p \vee q$ or $q \vee p$ (any q)
$\neg$ Intro / $\neg$ Elim	assumption p (resp. $\neg p$) leads to q and $\neg q$: write $\neg p$ (resp. p)
$\rightarrow$ Intro	subproof from p to q: write $p \rightarrow q$
$\vee$ Elim	from $p \vee q$ and subproofs $p \rightsquigarrow r$, $q \rightsquigarrow r$: write r
Derived	Modus Tollens, hypothetical syllogism, disjunctive syllogism, $\neg\neg$, De Morgan

Cheat sheet — Propositional logic

- A **logic** = syntax (well-formed formulas) + semantics (meaning, truth) + proof theory (rules to manipulate formulas).
- **Proposition**: a statement that is either true or false, never both. **Atomic** if its truth does not depend on other propositions.
- Connectives and their truth conditions: $p \wedge q$ true iff both true; $p \vee q$ true iff at least one true; $\neg p$ true iff p false; $p \rightarrow q$ **false only when** p **is true and** q **is false**; $p \leftrightarrow q$ true iff p and q have the same value.
- n atomic propositions $\Rightarrow 2^n$ rows in the truth table.
- **Valuation**: assigns T/F to each atomic proposition; each row of a truth table is a valuation.
- **Tautology**: true under every valuation. **Consistent** (satisfiable): true under at least one. **Inconsistent**: true under none.
- **Natural deduction**: Intro and Elim rule for each connective, plus Reit. $\neg$Intro, $\neg$Elim, $\rightarrow$Intro and $\vee$Elim need **subproofs** (temporary assumptions); lines inside a closed subproof cannot be reused.
- $\rightarrow$Elim = **Modus Ponens** ($p \rightarrow q, p \ / \ q$). Invalid look-alike: **affirming the consequent** ($p \rightarrow q, q \ / \ p$).
- Derived rules: Modus Tollens ($p \rightarrow q, \neg q \ / \ \neg p$), hypothetical syllogism, disjunctive syllogism, double negation, De Morgan ($\neg(p \wedge q) \equiv \neg p \vee \neg q$, $\neg(p \vee q) \equiv \neg p \wedge \neg q$).

3 First-order logic

[Slides · FOL: 1–21; Proof theory (FOL): 1–4]

Propositional logic treats "all monitors are ready" as one indivisible block, true or false. It cannot look *inside* the sentence and see that it talks about *all* objects of a certain kind. **First-order logic** (FOL, also called *predicate logic*) can: it has names for objects, functions, relations between objects and, above all, **quantifiers**

("for all", "there exists"). It is the logic behind Prolog and behind the beliefs of BDI agents.

This chapter covers, in the order of the slides:

1. why propositional logic is not enough;
2. the syntax of FOL: terms (constants, variables, functions) and predicates;
3. quantifiers ($\forall$, $\exists$, $\exists_1$) and how to translate English into FOL;
4. decidability: FOL has no truth-table method, and no method at all that always works;
5. proof theory for FOL: Frege/Hilbert-style axioms, soundness, the sequent calculus LK.

3.1 Why not propositional logic?

[Slides · FOL: 2–3]

Consider the argument:

$$\frac{\begin{array}{l}\text{all monitors are ready}\\ \text{X12 is a monitor}\end{array}}{\text{therefore X12 is ready}}$$

Intuitively it is *sound*: if we accept that the two premises (above the line) are true, we must accept that the conclusion is true too. Each of the three statements is a proposition (true or false), so we could try propositional logic:

$$p = \text{"all monitors are ready"}, \quad q = \text{"X12 is a monitor"}, \quad r = \text{"X12 is ready"}.$$

But then the argument becomes $p,\ q \ /\ \therefore\ r$, which is **not** valid: the valuation $p = q = \mathsf{T}$, $r = \mathsf{F}$ makes the premises true and the conclusion false. The sense of the argument is lost, because the connection between "monitors", "X12" and "ready" is hidden inside the atomic propositions.

Intuition — Propositional logic is too coarse-grained

Propositional logic sees sentences as black boxes. The validity of the monitor argument depends on what is *inside* the boxes: the same individual (X12), the same property (being a monitor, being ready), and the word "all". We need a logic that can name individuals, express properties and relations, and quantify over all individuals: first-order logic. In FOL the argument becomes

$$\forall x\,(\mathit{Monitor}(x) \to \mathit{Ready}(x)), \quad \mathit{Monitor}(X12) \ /\ \therefore\ \mathit{Ready}(X12)$$

and it is valid (Section 3.5).

Why this matters in practice

The slides come from a Software Engineering course, where FOL is used for **formal specification** (e.g. the Z notation): "every reactor has a temperature between 100 and 1000" is a requirement about *all* reactors. In AI the same language describes knowledge bases ("every employee has a manager"), database queries (SQL is essentially FOL), and the beliefs of agents.

3.2 Syntax of FOL: terms and predicates

[Slides · FOL: 4–13]

First-order logic is a generalisation of propositional logic with much greater expressive power. Its language has two layers:

- **terms** talk about *objects* (numbers, people, monitors);
- **predicates** (atoms) talk about *facts* on those objects, and are true or false.

Formulas are then built from atoms with the connectives of propositional logic and with quantifiers (Section 3.3).

3.2.1 Terms: constants, variables, functions

[Slides · FOL: 5–10]

Definition — Term (slide 5)

The basic components of FOL are **terms**. A term denotes some *object*, other than true or false.

The slides use a *typed* version of FOL (as in the Z specification language): every object belongs to a set, its *type* ($\mathbb{N}$, $\mathbb{Z}$, Person, Monitor, ...).

Definition — Constant and variable (slide 6)

- A **constant** of type T is a name that denotes some *particular* object in the set T. Examples: 8 (the number eight, in $\mathbb{N}$ and $\mathbb{Z}$); *mike* (the individual Michael Wooldridge); S after "let S be the set $\{1, 2, 3\}$" (by saying what it denotes we have given it an **interpretation**).
- A **variable** of type T is a name that can denote *any* value in the set T. A variable of type $\mathbb{N}$ can stand for any natural number.

Definition — Functional term (slides 7–8)

FOL has a set of **function symbols**; each denotes a function. Each function symbol has an **arity** $n \in \mathbb{N}$ (its number of arguments), an argument type for each argument and a return type. If f is a function symbol of type T, with arity n and

argument types $T_1, \ldots, T_n$, and $\tau_1, \ldots, \tau_n$ are terms of types $T_1, \ldots, T_n$, then

$$f(\tau_1, \ldots, \tau_n)$$

is a **functional term** (of type T).

This is just like a function in a programming language: the slides' Modula-2 example `PROCEDURE f(a1:T1; ... an:Tn) : T;` declares a function with n typed arguments and return type T.

Worked example — the function *plus* (slide 9)

Let *plus* take two natural numbers and return their sum. Then:

Acceptable terms	Not acceptable
$plus(2, 3)$	$plus(-1, 0)$ ($-1 \notin \mathbb{N}$: wrong type)
$plus(0, 1)$	$plus(0.1, 2)$ ($0.1 \notin \mathbb{N}$: wrong type)
$plus(plus(1, 2), 4)$	$plus(1)$ (wrong arity; added)
$plus(plus(plus(0, 1), 2), 4)$	

Terms can be nested: the argument of a function can itself be a functional term.

Writing $2 + 3$ instead of $plus(2, 3)$ is merely a convention (infix notation); for logic, $+, -, /, *, \sqrt{}, \sin, \cos, \ldots$ are all function symbols. For instance $(x + 3) * \sin 90$ is the term $times(plus(x, 3), \sin(90))$ (slide 10).

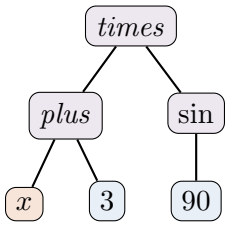


Figure 9: The term $(x+3)*\sin 90$ as a tree. Purple: function symbols; blue: constants; orange: variables. Leaves are always constants or variables.

3.2.2 Predicates

[Slides · FOL: 11–13]

Definition — Predicate (atom) (slide 11)

FOL contains a stock of **predicate symbols**, standing for relationships between objects. Each has an arity and a type for each argument. If P is a predicate symbol of arity n with argument types $T_1, \ldots, T_n$, and $\tau_1, \ldots, \tau_n$ are terms of those types, then

$$P(\tau_1, \ldots, \tau_n)$$

is a **predicate** (also called an **atom** or atomic formula), which will be either true or false under some interpretation.

Worked example — the predicate *gt* (slide 12)

Let gt ("greater than") take two natural numbers. Then $gt(4,3)$ is a predicate that evaluates to *true*, $gt(3,4)$ is a predicate that evaluates to *false*, while $gt(-1,2)$ is not a predicate at all (-1 has the wrong type). The usual mathematical relations $>, <, =, \geq, \leq, \neq, \in, \subset, \subseteq$ are predicate symbols written infix. Predicates combine with the connectives: $(2 > 3) \wedge (6 = 7) \vee (\sqrt{4} = 2)$.

Watch out — Terms vs. predicates: the most common confusion

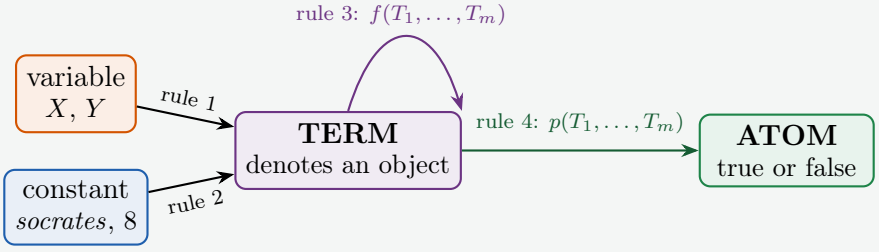
A **term** names an *object*; it is never true or false. A **predicate** states a *fact*; it is always true or false (under an interpretation). $plus(2,3)$ is a term (it denotes 5); $gt(plus(2,3),4)$ is a predicate (it is true). Writing $gt(4,3) \wedge 5$ is nonsense: you cannot "and" a number. Likewise $mother(ann)$ (a function: *the mother of Ann*, a person) differs from $mother(mary, ann)$ (a predicate: *Mary is the mother of Ann*, a fact).

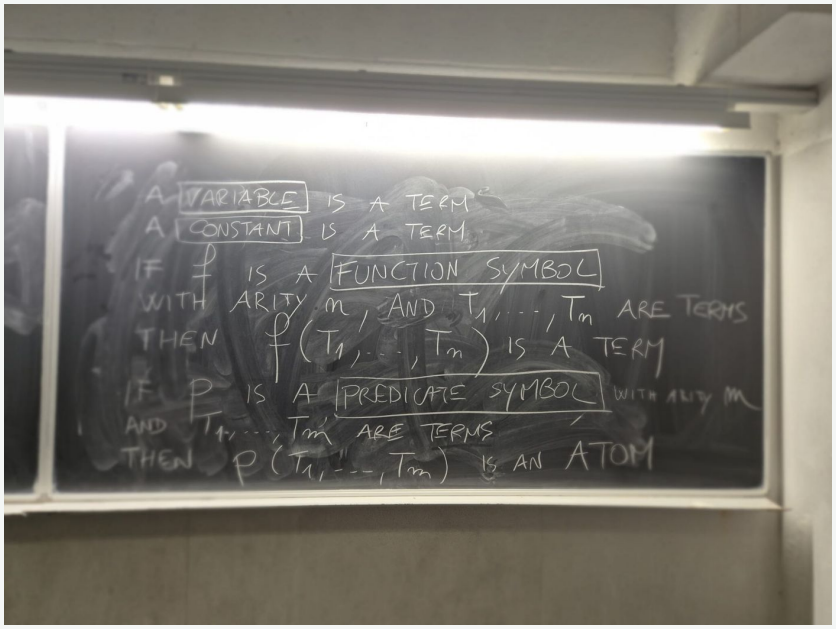
Some vocabulary (slide 13):

- a predicate with variables, e.g. $gt(x,3)$, cannot be evaluated until we know what x stands for (an interpretation of the variables);
- a predicate that contains **no variables** is a **proposition** (e.g. $gt(4,3)$);
- predicates of arity 1 are called **properties**, for example $Man(x)$, $Mortal(x)$, $Malfunctioning(x)$, all with one argument;
- predicates of arity 0 (no arguments) are called **primitive propositions**: they are exactly the atomic propositions of propositional logic. So propositional logic is the special case of FOL where all predicates have arity 0.

Whiteboard 1 — Terms and atoms written after the FOL lectures

Rule	Definition (as on the board)
1	A **variable** is a term.
2	A **constant** is a term.
3	If f is a **function symbol** with arity m, and $T_1, \ldots, T_m$ are terms, then $f(T_1, \ldots, T_m)$ is a term.
4	If p is a **predicate symbol** with arity m, and $T_1, \ldots, T_m$ are terms, then $p(T_1, \ldots, T_m)$ is an **atom**.





Original photo of the whiteboard

The definition is **recursive**: rule 3 builds terms out of terms, so terms can be nested as deeply as we want ($f(g(X), c)$). Rule 4 is applied *once*, on top: an atom contains terms, but an atom is never an argument of a function or of another predicate. Example with X a variable, *socrates* a constant, *father* a function of arity 1, *older* a predicate of arity 2: $father(socrates)$ is a term (rule 3), $older(father(socrates), X)$ is an atom (rule 4). Exactly the same definitions hold in Prolog (Section 7), where variables start with an upper-case letter, as on the board.

Why this matters in practice — A small knowledge base

[Not in the slides: added material]

A university information system could use: constants *alice*, *bob*, *sdai*; a function $advisor(x)$ (the advisor of student x); predicates $student(x)$, $enrolled(x, c)$, $passed(x, c, grade)$. The atom $enrolled(alice, sdai)$ is a row in a database table; $enrolled(advisor(bob), sdai)$ says that Bob's advisor attends SDAI. This is exactly how Prolog programs and agents' belief bases store knowledge.

3.3 Quantifiers

[Slides · FOL: 14–20]

Quantification is the central part of first-order logic. With terms and predicates alone we still cannot say:

- all men have a mother;
- every natural number has a prime factor.

These sentences speak about *all* objects, without naming them. We need quanti-

fiers.

3.3.1 The three quantifiers and their syntax

[Slides · FOL: 15–18]

Definition — Quantifiers (slide 15)

Symbol	Name	Read
$\forall$	universal quantifier	"for all …"
$\exists$	existential quantifier	"there exists …" (at least one)
$\exists_1$	unique quantifier	"there exists a unique …" (exactly one)

In the Z notation of the slides a quantified formula has the form

$$\text{quantifier} \quad \text{signature} \bullet \text{predicate}$$

where the signature is *variable : type*. Example: $\forall x : Man \bullet Mortal(x)$.

Worked example — reading quantified formulas (slides 17–18)

Formula	Meaning
$\forall x : Man \bullet Mortal(x)$	for all x of type Man, x is mortal (all men are mortal)
$\forall x : Man \bullet \exists_1 y :$ $Woman \bullet MotherOf(x, y)$	every man has exactly one mother (y is the mother of x)
$\exists m :$ $Monitor \bullet MonitorState(m, ready)$	there is a monitor in the ready state
$\forall r : Reactor \bullet \exists_1 t :$ $100..1000 \bullet Temp(r) = t$	every reactor has a temperature in the range 100 to 1000
$\exists n : \mathbb{N} \bullet n = n * n$	some natural number is equal to its own square (true: 0 and 1)
$\exists c : EC \bullet Borders(c, Albania)$	some EC country borders Albania (Greece)
$\forall m, n : Person \bullet \neg Superior(m, n)$	no person is superior to another
$\forall m : Person \bullet \neg\exists n :$ $Person \bullet Superior(m, n)$	ditto (same meaning)

Without types. Most AI texts (and Prolog) use untyped FOL: there is one big domain of objects, and types become ordinary predicates. The typed formulas translate with a fixed pattern that is worth memorising.

Key result — Typed to untyped quantifiers

[Not in the slides: added material]

$$\forall x : T \bullet P(x) \quad \text{becomes} \quad \forall x \left(T(x) \to P(x)\right)$$
$$\exists x : T \bullet P(x) \quad \text{becomes} \quad \exists x \left(T(x) \wedge P(x)\right)$$

$\forall$ goes with $\to$, $\exists$ goes with $\wedge$.

Watch out — The two classic translation mistakes

- "All men are mortal" is $\forall x\,(Man(x) \to Mortal(x))$, **not** $\forall x\,(Man(x) \wedge Mortal(x))$: the latter says that *everything* (every chair, every number) is a man and is mortal.
- "Some monitor is ready" is $\exists x\,(Monitor(x) \wedge Ready(x))$, **not** $\exists x\,(Monitor(x) \to Ready(x))$: the latter is true as soon as there exists any object that is not a monitor (false antecedent!), even if no monitor is ready.

Watch out — The order of quantifiers matters

[Not in the slides: added material]

$\forall x\,\exists y\,MotherOf(x, y)$: everybody has a mother (possibly a different one for each x): true.
$\exists y\,\forall x\,MotherOf(x, y)$: there is one y who is the mother of everybody: false. Swapping $\forall$ and $\exists$ changes the meaning; swapping two $\forall$ (or two $\exists$) does not.

3.3.2 Free and bound variables

[Not in the slides: added material]

A quantifier $\forall x$ or $\exists x$ **binds** the variable x inside its **scope** (the formula it is applied to). An occurrence of a variable that is not bound by any quantifier is **free**. In $\forall x\,(gt(x, y) \to gt(x, 0))$, x is bound and y is free. A formula with no free variables is a **sentence** (or *closed* formula): only sentences are simply true or false in an interpretation; a formula with free variables, like $gt(x, 3)$, needs a value for its free variables (slide 13). This distinction comes back in the proof rules of Section 3.5.

3.3.3 Quantifiers as big conjunctions and disjunctions

[Slides · FOL: 19–20]

Key result — Quantifiers over a finite set (slide 19)

Universal quantification is like conjunction, existential quantification like disjunction:

$$\forall n : \{2, 4, 6\} \bullet Even(n) \quad \text{is the same as} \quad Even(2) \wedge Even(4) \wedge Even(6)$$
$$\exists n : \{7, 8, 9\} \bullet Prime(n) \quad \text{is the same as} \quad Prime(7) \vee Prime(8) \vee Prime(9)$$

Key result — Duality of $\forall$ and $\exists$ (slide 20)

$$\begin{aligned} \forall x : T \bullet P(x) \quad &\leftrightarrow \quad \neg\, \exists x : T \bullet \neg P(x) \\ \exists x : T \bullet P(x) \quad &\leftrightarrow \quad \neg\, \forall x : T \bullet \neg P(x) \end{aligned}$$

Saying that everything has some property is the same as saying that there is nothing that does not have it. Saying that something has the property is the same as saying that it is not the case that everything lacks it.

Intuition — Duality is De Morgan in disguise

Over the finite set $\{a, b, c\}$:

$$\begin{aligned} \neg\, \forall x\, P(x) &= \neg\big(P(a) \wedge P(b) \wedge P(c)\big) && \triangleleft\ \forall = \text{big} \wedge \\ &\equiv \neg P(a) \vee \neg P(b) \vee \neg P(c) && \triangleleft\ \text{De Morgan} \\ &= \exists x\, \neg P(x) && \triangleleft\ \exists = \text{big} \vee \end{aligned}$$

Negating both sides gives the first duality law. Practical reading: to refute "all monitors are ready" you need one monitor that is *not* ready (a counterexample).

Key result — The unique quantifier, defined with the other two

[Not in the slides: added material]

$\exists_1 x\, P(x) \;\equiv\; \exists x \big(P(x) \wedge \forall y\, (P(y) \to y = x)\big)$: there is an x with the property, and any y with the property is that same x.

Exercise Q.1 — Translate into FOL (added)

Use the predicates $\mathit{Student}(x)$, $\mathit{Course}(x)$, $\mathit{Attends}(x, y)$, $\mathit{Passed}(x, y)$.

1. Every student attends some course.
2. Some course is attended by every student.
3. No student passed a course without attending it.
4. Not every student passed SDAI.

Solution

1. $\forall x \big(\mathit{Student}(x) \to \exists y\, (\mathit{Course}(y) \wedge \mathit{Attends}(x, y))\big)$
2. $\exists y \big(\mathit{Course}(y) \wedge \forall x\, (\mathit{Student}(x) \to \mathit{Attends}(x, y))\big)$ (stronger than 1: the order of quantifiers changed)
3. $\neg\exists x\, \exists y \big(\mathit{Student}(x) \wedge \mathit{Passed}(x, y) \wedge \neg \mathit{Attends}(x, y)\big)$, equivalently $\forall x\, \forall y \big(\mathit{Student}(x) \wedge \mathit{Passed}(x, y) \to \mathit{Attends}(x, y)\big)$ (by duality and De Morgan).
4. $\neg\forall x\, (\mathit{Student}(x) \to \mathit{Passed}(x, \mathit{sdai}))$, equivalently $\exists x\, (\mathit{Student}(x) \wedge \neg \mathit{Passed}(x, \mathit{sdai}))$: some student did not pass SDAI.

Lesson: always check the pattern $\forall \ldots \to$, $\exists \ldots \wedge$.

3.4 Decidability

[Slides · FOL: 21]

In propositional logic some formulas are tautologies (true under all interpretations), and there is a *procedure*, the truth-table method, that tells in a finite time whether any formula is a tautology.

Definition — Valid formula (slide 21)

A formula of FOL that is true under all interpretations is said to be **valid** (it is the FOL analogue of a tautology).

Truth tables do not work for FOL: an interpretation chooses a domain of objects (possibly infinite, like $\mathbb{N}$) and a meaning for every symbol, so there are infinitely many interpretations and a "truth table" would be infinite. Is there *any other* procedure guaranteed to tell us, in a finite amount of time, whether a FOL formula is or is not valid? **No.**

Key result — Undecidability of FOL (Church and Turing, 1936)

There is no algorithm that, given any FOL formula, always terminates and correctly answers whether the formula is valid. FOL is said to be **undecidable**.

Intuition — Semi-decidable: "yes" can be found, "no" may never come

[Not in the slides: added material]

FOL is *semi-decidable*: there are procedures (e.g. enumerating all proofs, or resolution) that, if the formula *is* valid, will find a proof in finite time. If the formula is *not* valid, the procedure may run forever, and at no point can you tell "not valid" apart from "valid, but the proof has not been found yet". You will meet exactly this behaviour in Prolog: a query either succeeds, fails, or loops forever (Section 7).

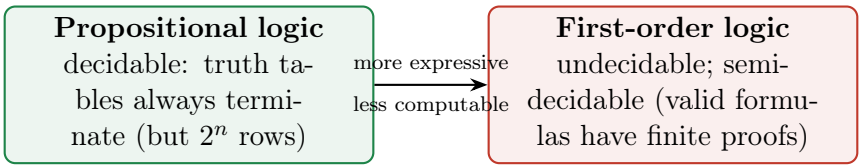


Figure 10: The price of expressive power: FOL can say much more, but no algorithm can decide validity for all its formulas.

Why this matters in practice

This is why practical symbolic AI uses *fragments* of FOL with better computational behaviour: Prolog uses Horn clauses with a specific search strategy (Section 7), description logics (behind ontologies and the Semantic Web) are decidable fragments, and agent languages such as AgentSpeak keep beliefs as ground atoms.

3.5 Proof theory for first-order logic

[Slides · Proof theory (FOL): 1–4]

The source for this part is a set of university lecture notes (Math 260A, UCSD, 2012), more technical than the rest of the course. The goal here is to understand the *ideas*: what a proof system for FOL looks like, what soundness means, and how quantifiers are handled.

In propositional logic the simplest proof system is the truth table. In FOL there is no analogue: the truth tables would be infinite (Section 3.4). We go directly to **Frege-style** systems, called **Hilbert-style** systems in FOL: *many axioms, very few rules.*

3.5.1 Frege (Hilbert-style) proofs

[Slides · Proof theory (FOL): 1–2]

Definition — Axioms of the Frege system for FOL (page 1)

- **Propositional axioms**: all axioms of propositional Frege systems, such as $\varphi \to (\psi \to \varphi)$, where now φ, ψ can be any FOL formulas.
- **Quantifier axioms**: for any formula φ and any term t,

$$\varphi(t) \to \exists x\, \varphi(x) \qquad\qquad \forall x\, \varphi(x) \to \varphi(t)$$

  (provided t is *substitutable* for x; the notes mostly ignore this technical condition).
- **Equality axioms** (if $=$ is in the language):
  - $\forall x\,(x = x)$ (reflexivity)
  - $\forall x \forall y\,(x = y \to y = x)$ (symmetry)
  - $\forall x \forall y \forall z\,(x = y \wedge y = z \to x = z)$ (transitivity)
  - $\forall \vec{x}\, \forall \vec{y}\,\big(x_1 = y_1 \wedge \cdots \wedge x_k = y_k \to f(x_1, \ldots, x_k) = f(y_1, \ldots, y_k)\big)$ for every function f of arity k
  - $\forall \vec{x}\, \forall \vec{y}\,\big(x_1 = y_1 \wedge \cdots \wedge x_k = y_k \to (P(x_1, \ldots, x_k) \to P(y_1, \ldots, y_k))\big)$ for every relation P of arity k (substitution of equals)

Watch out — Corrections to page 1 of the notes

- The notes say "t should be substitutable for t": it should read "for x".
- The last two equality axioms are printed without the hypothesis $x_1 = y_1 \wedge \cdots \wedge x_k = y_k$, which would make them say that f gives the same value on *any* arguments (false). The correct versions are written above: equal inputs give equal outputs, and equal objects have the same properties.

Intuition — Reading the quantifier axioms

$\forall x\,\varphi(x) \to \varphi(t)$: "if something holds for everything, it holds for any particular thing t" (*instantiation*). From "all humans are mortal" you may conclude "if Socrates is human, Socrates is mortal". $\varphi(t) \to \exists x\,\varphi(x)$: "if it holds for some particular t, then it holds for something" (*generalisation to* $\exists$). From $Ready(X12)$ conclude "some monitor is ready".

Key result — Rules of inference (pages 1–2)

Modus Ponens: $\dfrac{\varphi \qquad \varphi \to \psi}{\psi}$ $\qquad$ $\dfrac{\psi \to \varphi}{\psi \to \forall x\,\varphi}$ $\qquad$ $\dfrac{\varphi \to \psi}{\exists x\,\varphi \to \psi}$

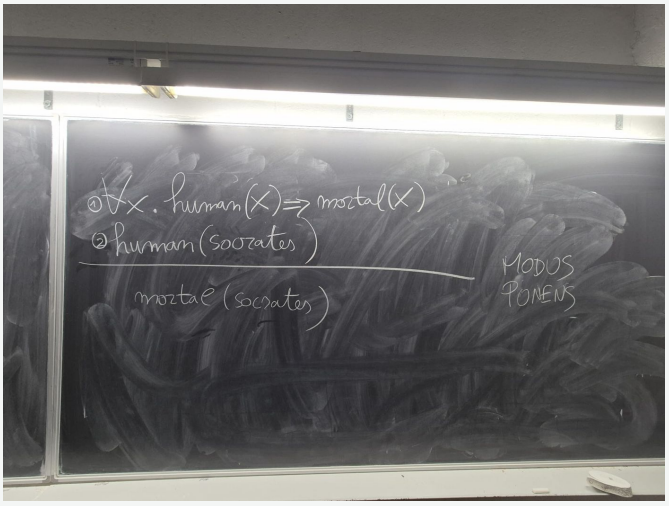
The two quantifier rules require that x **is not free in** ψ.

Intuition — Why "x not free in ψ"?

Rule 2: if $\psi \to \varphi(x)$ has been proved and ψ says nothing about x, then the proof worked for an arbitrary x, so $\psi \to \forall x\,\varphi$. Without the condition we could go from $x = 0 \to x = 0$ to $x = 0 \to \forall x\,(x = 0)$, i.e. "if this number is 0 then every number is 0": wrong. Rule 3: if $\varphi(x)$ implies ψ whatever x is, then knowing that *some* x satisfies φ is enough to get ψ, provided ψ itself does not depend on which x it was.

Whiteboard 2 — Modus Ponens: Socrates is mortal written during the FOL proof-theory lecture

$$\dfrac{\begin{array}{ll}① & \forall x\,.\ human(x) \Rightarrow mortal(x) \\ ② & human(socrates)\end{array}}{mortal(socrates)} \quad \textsc{Modus Ponens}$$



Original photo of the whiteboard

The board shows Aristotle's syllogism (Section 1.3) in FOL. Strictly speaking, Modus Ponens needs an implication whose antecedent is *exactly* $human(socrates)$,

so a quantifier step comes first. In full, with the Frege system above: Writing h for *human*, m for *mortal* and s for *socrates*:

1	$\forall x\,(h(x) \to m(x))$	premise
2	$h(s)$	premise
3	$\forall x\,(h(x) \to m(x)) \to (h(s) \to m(s))$	quantifier axiom with $t = s$
4	$h(s) \to m(s)$	Modus Ponens 1, 3
5	$m(s)$	Modus Ponens 2, 4

In Prolog the same knowledge is `mortal(X) :- human(X).` and `human(socrates).`; the query `?- mortal(socrates).` answers `true`: Prolog's resolution does the instantiation ($X = socrates$, called *unification*) and the Modus Ponens step in one go (Section 7).

3.5.2 Provability and soundness

[Slides · Proof theory (FOL): 2]

Definition — Provability $\Gamma \vdash \varphi$ (page 2)

Let Γ be a set of formulas (the hypotheses). We write $\Gamma \vdash \varphi$ ("Γ proves φ") if there is a finite sequence of formulas, each coming from Γ, from the axioms, or from earlier formulas by an inference rule, ending at φ.

Recall that $\Gamma \models \varphi$ ("Γ entails φ") means: in every structure (interpretation) M in which all formulas of Γ are true, φ is true too. $\vdash$ is about *symbols and rules*; $\models$ is about *meaning and truth*.

Key result — Soundness theorem (page 2)

If $\Gamma \vdash \varphi$, then $\Gamma \models \varphi$. (Everything provable is true.)

Proof idea. Take a proof P of φ from Γ and a structure M in which Γ is true, i.e. for all object assignments σ (values for the free variables) and all $\gamma \in \Gamma$, $M \models \gamma[\sigma]$. Show by **induction on the number of lines** of P that every line ψ is true in M under all σ. Base cases: hypotheses (true by assumption) and axioms (true in every structure). Modus Ponens preserves truth (truth table of $\to$). For the $\exists$-rule $\frac{\varphi \to \psi}{\exists x \varphi \to \psi}$ argue by contrapositive:

1. suppose $M \not\models (\exists x \varphi \to \psi)[\sigma]$; then $M \models \exists x \varphi[\sigma]$ and $M \not\models \psi[\sigma]$;
2. the first fact gives an object m of M with $M \models \varphi[\sigma(m/x)]$ (σ changed so that x denotes m);
3. let $\sigma' = \sigma(m/x)$. Since x is not free in ψ, changing the value of x does not affect ψ, so $M \not\models \psi[\sigma']$;
4. under σ', φ is true and ψ false, so $M \not\models (\varphi \to \psi)[\sigma']$: the premise of the rule is not true under all assignments. Contrapositive: if the premise is, so is the conclusion.

(The notes write $M \models \psi[\sigma(m/x)]$ in step 2: it should be φ.) Note that we needed to

change the object assignment to make this work.

Key result — Completeness (Gödel, 1929)

[Not in the slides: added material]

The converse also holds: if $\Gamma \models \varphi$ then $\Gamma \vdash \varphi$. So for FOL, "provable" and "true in all models" coincide. This does not contradict undecidability: a proof always *exists* for valid formulas, but there is no algorithm that also tells you, in finite time, when *no* proof exists.

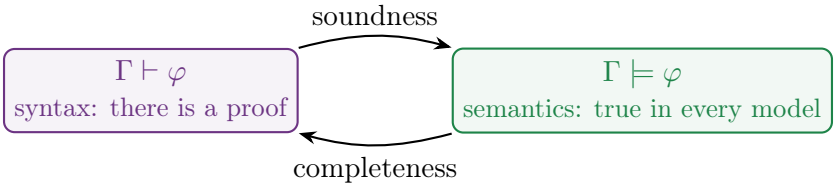


Figure 11: Soundness: the rules never prove something false. Completeness: every truth that follows has a proof.

3.5.3 Sequent calculus LK

[Slides · Proof theory (FOL): 2–4]

A second style of proof system, invented by Gentzen: the **sequent calculus**. A **sequent** has the form

$$\Gamma \to \Delta \qquad (\Gamma, \Delta \text{ finite lists of formulas})$$

and means: *if all formulas in* Γ *are true, then at least one formula in* Δ *is true.* LK (from the German *Logischer Kalkül*) has the opposite philosophy of Frege systems: *one axiom, many rules.*

Variable conventions. $a, b, c, \ldots$ are used only for *free* variables and $x, y, z, \ldots$ only for *bound* variables. Consequence: a sub-formula of a formula is not always a formula. In $\forall x\,(x \leq a)$, the part $x \leq a$ is a *semi-formula* (x is not bound there); similarly $x + a$ is a *semi-term*. With these conventions any term t is always substitutable for a free variable a in a formula φ.

Key result — LK (page 3)

Initial sequents: $A \to A$, for any formula A.
Rules: all rules of the propositional sequent calculus, plus four quantifier rules:

$$\forall\text{-right}\ \frac{\Gamma \to \Delta, A(b)}{\Gamma \to \Delta, \forall x A(x)} \qquad \exists\text{-right}\ \frac{\Gamma \to \Delta, A(t)}{\Gamma \to \Delta, \exists x A(x)}$$

$$\exists\text{-left}\ \frac{A(b), \Gamma \to \Delta}{\exists x A(x), \Gamma \to \Delta} \qquad \forall\text{-left}\ \frac{A(t), \Gamma \to \Delta}{\forall x A(x), \Gamma \to \Delta}$$

In $\forall$-right and $\exists$-left the free variable b is an **eigenvariable**: it must not appear

anywhere in the lower sequent (the substitution x/b "captures" all occurrences of b). In $\exists$-right and $\forall$-left, t is *any* term and may occur in the lower sequent.

Intuition — Eigenvariable = "an arbitrary object we know nothing about"

- $\forall$-*right*: to prove "for all x, $A(x)$", prove $A(b)$ for a *fresh* b about which the hypotheses say nothing. Like in maths: "let b be an arbitrary natural number…". If b appeared in Γ (e.g. Γ says $b = 0$), it would not be arbitrary.
- $\exists$-*left*: if we know "some x has A", we may give it a fresh name b, but we cannot assume it is any object we already know.
- $\exists$-*right* and $\forall$-*left*: to show "something has A", exhibit any witness t; to use "everything has A", instantiate with any t.

3.5.4 LKe: adding equality

[Slides · Proof theory (FOL): 4]

LK has no rules for $=$. **LKe** adds five initial sequents (for any terms s, t, u, s_i, t_i):

- $\to s = s$
- $s = t \to t = s$
- $s = t,\ t = u \to s = u$
- $s_1 = t_1, \ldots, s_k = t_k \to f(s_1, \ldots, s_k) = f(t_1, \ldots, t_k)$ for any *function symbol* f of arity k
- $s_1 = t_1, \ldots, s_k = t_k,\ P(s_1, \ldots, s_k) \to P(t_1, \ldots, t_k)$ for any relation P of arity k

(The notes write "for any formula f": f is a function symbol. The last sequent is printed with a second arrow; the version above is the intended one.)

Worked example — proving symmetry of equality in LKe (page 4)

Goal: $\to \forall x \forall y\,(x = y \to y = x)$. Read the proof bottom-up as a plan, top-down as a derivation:

$$\dfrac{\dfrac{\dfrac{a = b \to b = a}{\to a = b \to b = a}\,{\to}\text{-right}}{\to \forall y\,(a = y \to y = a)}\,\forall\text{-right }(b \text{ fresh})}{\to \forall x \forall y\,(x = y \to y = x)}\,\forall\text{-right }(a \text{ fresh})$$

1. Start from the LKe initial sequent $a = b \to b = a$ (symmetry for the terms a, b).
2. $\to$-right (a propositional rule: from $A, \Gamma \to \Delta, B$ infer $\Gamma \to \Delta, A \to B$) moves $a = b$ to the right as an antecedent.
3. $\forall$-right with eigenvariable b: b does not occur in the lower sequent $\to \forall y\,(a = y \to y = a)$, so the condition holds.
4. $\forall$-right again with eigenvariable a.

3.5.5 Natural deduction rules for quantifiers

[Not in the slides: added material]

To connect with the natural deduction of Section 2.4, here are the four quantifier rules in the same Intro/Elim style (same ideas as LK):

$\forall$ Elim	from $\forall x\,\varphi(x)$ write $\varphi(t)$ for any term t (instantiation)
$\forall$ Intro	if you derived $\varphi(b)$ for a fresh b appearing in no open assumption, write $\forall x\,\varphi(x)$
$\exists$ Intro	from $\varphi(t)$ write $\exists x\,\varphi(x)$
$\exists$ Elim	from $\exists x\,\varphi(x)$, assume $\varphi(b)$ with b fresh in a subproof; if you derive ψ (not containing b), write ψ outside

Worked example — the monitor argument, proved

1	$\forall x\,(\mathit{Monitor}(x) \to \mathit{Ready}(x))$	Premise
2	$\mathit{Monitor}(X12)$	Premise
3	$\mathit{Monitor}(X12) \to \mathit{Ready}(X12)$	$\forall$ Elim: 1
4	$\mathit{Ready}(X12)$	$\to$ Elim: 3, 2

The argument of Section 3.1, which propositional logic could not capture, becomes a two-step proof.

Cheat sheet — First-order logic

- **Terms** denote objects: constants (*socrates*, 8), variables (x), functional terms $f(t_1, \ldots, t_n)$ ($\mathit{plus}(2, 3)$, $\mathit{mother}(x)$). A term is never true or false.
- **Atoms** (predicates): $P(t_1, \ldots, t_n)$, true or false under an interpretation. Arity 1 = *property*; arity 0 = *primitive proposition*; an atom without variables is a *proposition*.
- Formulas: atoms combined with $\neg, \wedge, \vee, \to, \leftrightarrow$ and quantifiers $\forall x$ (for all), $\exists x$ (there exists), $\exists_1 x$ (exactly one).
- Translation pattern: "all A are B" = $\forall x\,(A(x) \to B(x))$; "some A is B" = $\exists x\,(A(x) \wedge B(x))$.
- $\forall$ over a finite set = big $\wedge$; $\exists$ = big $\vee$. Duality: $\forall x\,P(x) \equiv \neg\exists x\,\neg P(x)$, $\exists x\,P(x) \equiv \neg\forall x\,\neg P(x)$.
- **Valid** = true under all interpretations. Validity in FOL is **undecidable** (no algorithm always answers yes/no).
- $\Gamma \vdash \varphi$: φ is provable from Γ (syntax, rules). $\Gamma \models \varphi$: φ is true in every model of Γ (semantics). **Soundness**: $\Gamma \vdash \varphi \Rightarrow \Gamma \models \varphi$.
- Frege/Hilbert system: many axioms + Modus Ponens + two quantifier rules. Sequent calculus LK: one axiom $A \to A$ + many rules; $\forall$-right and $\exists$-left need a fresh *eigenvariable*.
- Socrates (Whiteboard 2): instantiate $\forall x(\mathit{human}(x) \to \mathit{mortal}(x))$ with $x = \mathit{socrates}$, then Modus Ponens.

Part III

Agents and Multi-Agent Systems

4 Agents and multi-agent systems

[Slides · Intro to agents & MAS: 1–66]

Part I defined AI as the study of *intelligent agents*. This chapter makes the word *agent* precise, defines *multi-agent systems* (MAS), explains the three ways in which MAS are used (to *design* software, to *implement* it, to *simulate* the world), and tells the history of the idea, from AI planning and objects to intentional systems, modal logic, the Contract Net Protocol and actors.

4.1 What is an agent?

[Slides · Intro to agents & MAS: 5–11]

There is no single definition of agent, but two families of definitions are standard.

4.1.1 The weak definition

[Slides · Intro to agents & MAS: 5–8]

Definition — Agent, weak notion (Jennings, Sycara, Wooldridge, 1998)

An agent is a hardware or software system that is

- **situated**,
- **autonomous**,
- **flexible**, which means:
  - **reactive**,
  - **proactive**,
  - **social**.

The slides only list the words; here is what each one means (from the 1998 paper by N. Jennings, K. Sycara and M. Wooldridge, whose photos are on slide 8).

Property	Meaning	Robot vacuum cleaner
situated	it lives *in* an environment: it receives sensory input from it and acts to change it	senses walls, dirt and stairs; moves and sucks
autonomous	it acts without the direct intervention of humans or other agents, and has control over its own actions and internal state	nobody drives it: it decides where to go next
reactive	it perceives the environment and responds *in a timely fashion* to the changes that occur in it	a chair is moved: it turns to avoid it
proactive	it does not simply act in response to the environment: it exhibits *goal-directed* behaviour and takes the initiative	when the battery is low, it goes back to the dock on its own initiative
social	it interacts with other agents (and humans) to complete its tasks and to help others	coordinates with the mop robot so they don't clean the same room

Table 10: The properties of the weak definition, explained with one everyday agent.

Intuition — The hard part is the balance

Being only reactive is easy (a thermostat), being only proactive is easy (a program that blindly executes a plan). The difficulty, and the reason why agent architectures exist (Section 5), is to be *both*: pursue goals over time, while still reacting quickly when the world changes and the plan no longer makes sense.

Worked example — is it an agent?

[Not in the slides: added material]

System	Verdict
a pocket calculator	no: not situated in an environment it perceives, no autonomy, no goals of its own
a thermostat	a degenerate agent: situated, autonomous, reactive, but not proactive nor social
a spam filter	situated (in the mail stream), autonomous and reactive; weakly proactive
a negotiating bot on an e-commerce platform	yes: it pursues the goal of buying cheaply (proactive), reacts to offers (reactive) and negotiates with sellers' bots (social)

4.1.2 The strong definition

[Slides · Intro to agents & MAS: 9–11]

Besides the properties of the weak definition, an agent may be conceptualised following an **anthropomorphic** approach (the **strong** definition): we describe and build it using notions that we normally apply to humans.

Definition — Agent, strong notion

- **Mentalistic notions** (Y. Shoham, *Artificial Intelligence* 1993; A. S. Rao and M. P. Georgeff, KR&R 1992): the agent has *beliefs*, *desires*, *intentions*, *commitments*, ...
- **Emotional notions** (J. Bates, *Communications of the ACM* 1994): friendliness, trust, distrust; agents must be *believable*.

Worked example — a strong agent: a personal travel assistant

[Not in the slides: added material]

- *belief*: "the 8:15 train to Milan is cancelled";
- *desire*: "my user should be in Milan before 11:00";
- *intention*: "book the 8:45 train" (the desire it committed to, with a plan);
- *commitment*: having promised the user to handle the trip, it keeps trying alternatives (a bus, a car-sharing ride) until the trip is booked or impossible.

Beliefs, desires and intentions are exactly the three components of the **BDI** architecture (Section 8).

Why this matters in practice — Believable agents

Emotional notions matter wherever agents interact with people: virtual characters in video games and training simulations, conversational assistants, social robots in elderly care. A character that reacts with plausible emotions is "believable", and users trust it (or distrust it) accordingly.

4.2 What is a multi-agent system?

[Slides · Intro to agents & MAS: 12–17]

A single agent is rarely enough: interesting problems require several agents, each with its own goals and abilities, that interact. Such a system is a **multi-agent system** (MAS). Slide 12 shows the classic picture, redrawn in Figure 12.

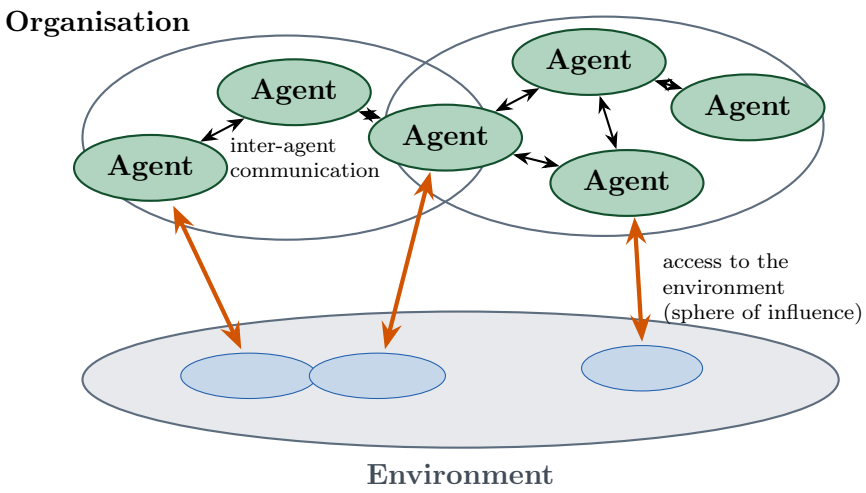


Figure 12: Structure of a MAS (slide 12). Agents (green) communicate with each other, are grouped in organisations (grey ellipses) and each one can perceive and act on its own portion of the shared environment (blue regions, orange arrows). Spheres of influence may overlap: two agents may act on the same part of the environment.

Definition — Characteristics of a MAS (slides 14–17)

In a multi-agent system:

1. each agent has **incomplete information**, or capabilities for solving the problem, thus each agent has a **limited viewpoint**;
2. there is **no global system control**;
3. data is **decentralised**;
4. computation is **asynchronous**.

Worked example — a smart-city ride-sharing MAS

[Not in the slides: added material]

Passenger agents (one per user app), driver agents (one per car) and a pricing agent.

1. *Limited viewpoint*: a driver agent knows its own position and route, not the positions of the other cars or the requests of all passengers.
2. *No global control*: no central program decides every move; drivers accept or refuse rides.
3. *Decentralised data*: each phone stores its own user's preferences and history.
4. *Asynchronous computation*: requests and answers arrive at any time; nobody waits for a global "clock tick".

The same four features describe e-commerce marketplaces, power grids with home batteries, fleets of warehouse robots and swarms of drones.

Intuition — Why "no global control" is a feature, not a bug

A central controller is a single point of failure and a bottleneck, and in an *open* system (agents of different owners, joining and leaving at any time) it simply cannot exist: nobody owns all the agents. MAS replace central control by *interaction*: negotiation, coordination, competition. This is why agent *communication* (Section 9) is a central topic of the course.

4.3 Three ways of using MAS

[Slides · Intro to agents & MAS: 18–41]

Agents and MAS can enter a software project at three different levels:

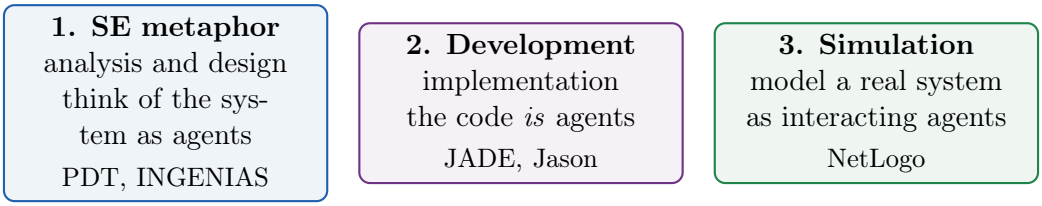


Figure 13: The three uses of MAS (slides 18–41). They can be combined: e.g. design with agents and implement with mainstream technologies.

4.3.1 MAS as a software engineering metaphor

[Slides · Intro to agents & MAS: 18–21]

Key result — Zambonelli and Omicini (JAAMAS, 2004)

"...today's software engineering approaches are increasingly adopting abstractions approaching that of agent-based computing. This trend can be better understood by recognising that *the vast majority of modern distributed systems scenarios are intrinsically prone to be developed in terms of MASs*, and that modern distributed systems are already de facto MASs, i.e., they are indeed composed of autonomous, situated, and social components."

In other words: even if you never write a line of "agent code", thinking in terms of agents (who are the autonomous components? what are their goals? how do they interact?) helps in the *analysis* and *design* of distributed systems. Tools exist to support this: **PDT** (Prometheus Design Tool, RMIT University) and **INGENIAS**, which let you draw agents, goals, messages and data stores as diagrams. Methodologies are the topic of Section 6.

4.3.2 Agent-based development

[Slides · Intro to agents & MAS: 22–36]

Definition — Agent-based development (slides 22–23)

Agents are employed as the technology for **implementing the software application itself**. This choice is usually made when

- the agent metaphor was already applied in the earlier engineering steps (analysis, design);
- the intelligence of the *individual* agent is more relevant than the "emerging intelligence" of the overall system;
- complex patterns of interaction may be required;
- the application is geographically **distributed**, **decentralised**, or **open**.

Two agent platforms are shown: **JADE** (Java Agent DEvelopment framework, Section 10), whose sniffer tool draws the messages exchanged by agents as a sequence diagram (slide 24), and **Jason** (slide 25, an interpreter of AgentSpeak, see the book *Programming multi-agent systems in AgentSpeak using Jason*; Section 8). (Slide 25 is labelled "Jade" but refers to Jason.)

Watch out — Why industry rarely uses agent platforms (slides 26–27)

Many agent-based development tools exist, but they are *not commercial projects supported by large and solid companies*. This is one of the reasons for the limited adoption of this technology by industry. A pragmatic answer: use technologies that are not officially tagged as "agent-oriented", but are very suitable for implementing software agents, in the *implementation* stage, and a "pure" agent approach for the *modelling and design* stages.

The slides then list research works that follow this idea, mapping agents onto mainstream technologies over the years:

Year	Technology	Work shown on the slide
2009	service-oriented computing	*Agents and Service-Oriented Computing for Autonomic Computing: A Research Agenda*
2011	cloud	*Software Agents as Cloud Computing Services*; *Cloud Computing and Software Agents: Towards Cloud Intelligent Services*
2013	web services	JADE Web Services Integration Gateway (WSIG)
2013	Internet of Things	*From Internet of Things to Internet of Agents*
2015	Internet of Things	*Agents of Things (AoT): an intelligent operational concept of the IoT*
2018	IoT and the Web	EMAS 2018 keynote: *Autonomous Agents for Flexible Hypermedia Systems*
2023	virtual reality	*VEsNA*: virtual environments via natural language agents, applied to factory automation (Gatti, Mascardi)
2024	extended reality	XRIA 2024: first workshop on "eXtended Reality & Intelligent Agents" at ECAI

Table 11: Agents implemented with mainstream technologies (slides 28–36).

Why this matters in practice — Agents everywhere, under other names

A microservice that watches a queue, decides and calls other services is an agent in disguise; so is an IoT device that negotiates energy prices, or a non-player character in a VR training environment that understands spoken commands. The agent *concepts* (autonomy, goals, interaction) transfer to these technologies even when no agent platform is used.

4.3.3 MAS for simulation

[Slides · Intro to agents & MAS: 37–41]

Key result — Agent-based modelling and simulation (Macal and North, 2007)

"Agent-based modeling and simulation (ABMS) is a new approach to modeling systems comprised of autonomous, interacting agents. ABMS promises to have far-reaching effects on the way that businesses use computers to support decision-making and researchers use electronic laboratories to support their research."
"Some have gone so far as to contend that ABMS is a *third way of doing science*, in addition to traditional deductive and inductive reasoning. [...] Applications range from modeling agent behavior in the stock market, supply chains, and consumer markets, to predicting the spread of epidemics, the threat of bio-warfare, and the factors responsible for the fall of ancient civilizations."

The tool shown on slide 41 is **NetLogo**, a programmable environment where thousands of simple agents ("turtles") move on a grid of "patches" and the global behaviour is observed in real time. ABMS is the topic of Section 11.

Intuition — Deduction, induction, simulation

Deduction: from general laws to particular consequences (logic, Part II). Induction: from many observations to general laws (statistics, machine learning). Simulation: write the *local* rules of each individual, run them, and *observe* what global phenomenon emerges (a traffic jam, an epidemic wave, a market crash). That is the "third way".

4.4 A short history of agents and MAS

[Slides · Intro to agents & MAS: 42–66]

Current interest in autonomous agents did not emerge from a vacuum. The main contributing disciplines are **artificial intelligence**, **object-oriented programming** (and concurrent object-based systems) and **human-computer interface design** (slide 43).

4.4.1 From artificial intelligence

[Slides · Intro to agents & MAS: 44–47]

1961–71 AI **planning**: Newell and Simon (1961), Fikes and Nilsson's STRIPS (1971). Until the 1980s, AI studies the components of intelligence (learning, reasoning, vision…) *in isolation*; expert systems are "disembodied intelligence": no real-time capabilities, no social ability.

1987 Disillusion with **first-principle planning** (Chapman): building a new plan from scratch for every goal is too slow to be reactive ("calculative rationality") and undecidable in many cases. Georgeff and Lansky propose the **BDI** architecture for practical reasoning.

1991 Brooks: **behavioural** or **reactive AI**, intelligence without symbolic representation.

1992–94 **Hybrid architectures**, usually stratified in layers: TouringMachines (Ferguson, 1992), InteRRaP (Müller and Pischel, 1994).

1993 Shoham, *Agent-oriented programming*: agents are not just a branch of AI, but an autonomous technological and methodological approach. Shoham is considered the father of agent-oriented programming.

Figure 14: From AI to agents (slides 44–47). The architectures named here (reactive, deliberative, hybrid, BDI) are the subject of Section 5.

Definition — Practical reasoning agents (slide 46)

Agents whose architecture is modelled on, or inspired by, a theory of **practical reasoning** in humans: the kind of pragmatic reasoning that we use to decide *what to do* (as opposed to theoretical reasoning, which decides what is true). The most widely known practical reasoning architecture is **Belief-Desire-Intention** (BDI).

4.4.2 From object-oriented programming

[Slides · Intro to agents & MAS: 48–49]

Objects are computational entities that *encapsulate some state*, are able to *perform actions* (methods) on this state, and *communicate by message passing*. This sounds very much like an agent. So, what is the difference?

	Object	Agent
Autonomy	controls its *state* (private fields), but not its *behaviour*: if another object calls a public method, the method runs	controls its state *and* its behaviour: a request is a message the agent may refuse
Motto	"Objects do it for free; agents do it for money."	
Flexibility	the standard object model says nothing about how to build systems that integrate proactive, reactive and social behaviour	reactive, proactive and social behaviour are part of the model
Thread of control	usually passive: acts only when called	has its own thread of control: always running, takes the initiative

Table 12: Agents vs. objects (slide 49; last row added).

Worked example — the same request, two worlds

[Not in the slides: added material]

Object: `account.withdraw(100)` *always* executes the method (perhaps throwing an exception). Agent: Alice's banking agent *receives* the message "please transfer 100 euro to Bob", checks its own goals ("keep at least 500 on the account", "do not pay unknown parties") and may answer "refuse". One could argue that we can program this with objects too; the point (slide 49) is that the object model has nothing to say about it, while for agents it is the core of the abstraction.

4.4.3 From human-computer interaction

[Slides · Intro to agents & MAS: 50]

Instead of *direct manipulation* (the user does everything by clicking), we would like programs that behave like **"digital butlers"** (N. Negroponte, *Being Digital*, 1995):

- "The agent answers the phone, recognizes the callers, disturbs you when appropriate, and may even tell a white lie on your behalf."
- "If you have somebody who knows you well and shares much of your information, that person can act on your behalf very effectively. If your secretary falls ill, it would make no difference if the temping agency could send you Albert Einstein. This issue is not about IQ. It is shared knowledge and the practice of using it in your best interests."
- "Like an army commander sending a scout ahead … you will dispatch agents to collect information on your behalf."

Thirty years later, personal assistants and AI agents that read your e-mail and book your travel are trying to fulfil exactly this vision.

4.4.4 Agents as intentional systems

[Slides · Intro to agents & MAS: 51–60]

When explaining human activity we often say things like:

- Janine took her umbrella because she *believed* it was going to rain;
- Michael worked hard because he *wanted* to possess a PhD.

These statements use a **folk psychology**: human behaviour is predicted and explained by attributing attitudes such as believing, wanting, hoping, fearing. These attitudes are called the **intentional notions**.

Definition — Intentional system (D. C. Dennett, *The Intentional Stance*, 1989)

An **intentional system** is a system made up of entities whose behaviour can be predicted by the method of attributing *beliefs*, *desires* and *rational acumen*.

Intuition — The intentional stance in practice

You do not know the code of a chess program, but you predict its moves by saying "it *wants* to win and *believes* my queen is unprotected, so it will take it". That works much better than reasoning about transistors. Taking the intentional stance towards a system is useful when it is too complex to explain mechanically, yet behaves rationally.

If we adhere to the *strong* definition of agents, intentional systems are a suitable theory for agents (slides 56–60). Representing intentional notions raises delicate syntactic and semantic questions, and the usual answers are:

- **modal logic** languages are suitable for *specifying* agents as intentional systems;
- languages based on **computational (modal) logic** are suitable for *programming* agents;
- axiomatised logic-based languages can undergo **axiomatic verification**; other languages can be used for **model checking**.

Definition — Modal logic (slides 61–63)

Modal logic extends classical logic with a new connective $\Box$ (**necessity**) and its derivable counterpart $\Diamond$ (**possibility**).

- $\Box p$: p is *necessarily* true, i.e. true in every possible scenario ("possible world");
- $\Diamond p$: p is *possibly* true, i.e. true in at least one possible scenario.

$\Diamond$ can be defined from $\Box$: $\Diamond p \leftrightarrow \neg\Box\neg p$.

"p is possible" = "it is not necessary that p is false". It is the same duality as $\exists$ and $\forall$ (Section 3.3): $\Box$ is a "for all worlds", $\Diamond$ a "there exists a world".

Modal logic	$\Box p$ reads	Example
epistemic	the agent knows/believes p	$B_{robot}\ door_open$: the robot believes the door is open
temporal	p always holds (in the future)	$\Box\neg collision$: the drones never collide
deontic	p is obligatory	$O(pay_taxes)$; $\Diamond$ becomes "permitted"
dynamic	after every execution of action a, p holds	$[open]\ door_open$

Table 13: Kinds of modal logics (slide 64; readings and examples added). Combinations exist: **BDI logic** combines beliefs, desires, intentions and time; **KARO** combines knowledge, abilities, results and opportunities.

4.4.5 Two ancestors of MAS: Contract Net and Actors

[Slides · Intro to agents & MAS: 65–66]

Definition — Contract Net Protocol (Davis and Smith, 1983)

Agents can dynamically take two **roles**: **manager** or **contractor**.

1. Given a task, an agent first determines whether it can break it into subtasks that could be performed concurrently.
2. As *manager*, it uses the protocol to **announce** the tasks that could be transferred, and requests **bids** from nodes that could perform them.
3. A node that receives an announcement replies with a **bid**, indicating how well it thinks it can perform the task.
4. The manager collects the bids and **awards** the task to the best bidder, which becomes a *contractor*.

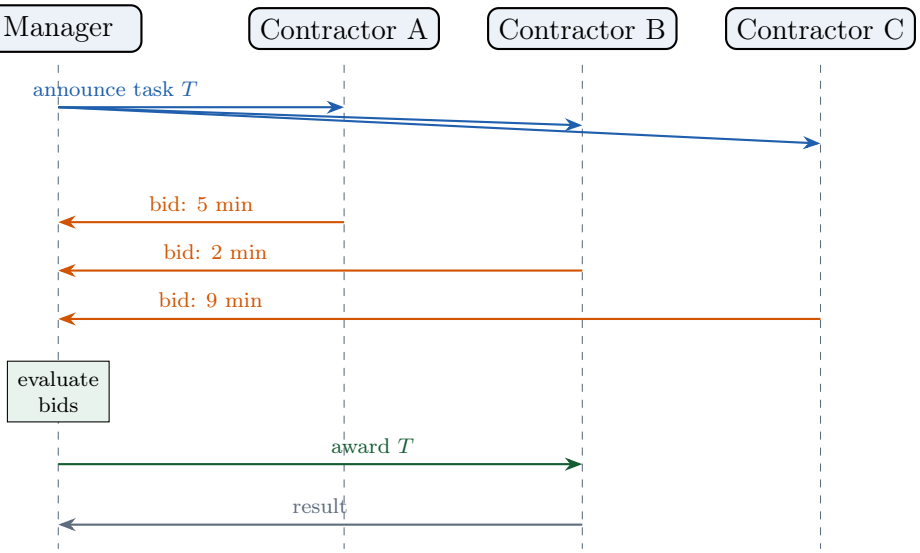


Figure 15: A run of the Contract Net Protocol: one announcement, three bids, the task is awarded to B (the best bid). The same protocol appears in FIPA as an interaction protocol (Section 9).

Watch out — Slide 65: who collects the bids?

The slide says "*the contractor* collects the bids and awards the task to the best bidder". It is the **manager** who announces the task, collects the bids and awards the task; the winner then becomes the contractor.

Why this matters in practice

Contract Net is everywhere where work is distributed by auction: a cloud scheduler choosing the server that can run a job fastest, a logistics platform assigning a delivery to the nearest courier, a factory controller assigning a part to the least loaded machine.

Definition — Actors (Hewitt, Bishop, Steiger, IJCAI 1973; Agha, 1986)

Actors were proposed as universal primitives of concurrent computation: self-contained, interactive, autonomous components of a computing system that communicate by **asynchronous message passing**. The basic actor primitives are:

- **create**: create an actor from a behaviour description and a set of parameters (possibly including existing actors);
- **send**: send a message to an actor;
- **become**: change an actor's local state.

Intuition — Actors today

Each actor has a mailbox, processes one message at a time, and can only affect others by sending messages: no shared memory, so no locks. Erlang (used for telecom switches and WhatsApp's servers) and the Akka framework are built on actors. Agents add to actors *goals* and *reasoning*; the messaging core is the same, and JADE agents (Section 10) work exactly like this.

Cheat sheet — Agents and MAS

- **Weak definition** (Jennings, Sycara, Wooldridge 1998): an agent is a hardware or software system that is **situated**, **autonomous** and **flexible**, i.e. **reactive**, **proactive** and **social**.
- **Strong definition**: anthropomorphic; *mentalistic* notions (beliefs, desires, intentions, commitments: Shoham 1993, Rao & Georgeff 1992) and *emotional* notions (believable agents: Bates 1994).
- **MAS**: each agent has incomplete information/capabilities (limited viewpoint); no global control; decentralised data; asynchronous computation. Agents live in organisations and act on a shared environment.
- Three uses: MAS as a **software-engineering metaphor** (analysis/design:

PDT, INGENIAS); **agent-based development** (implementation: JADE, Jason; or mainstream tech such as SOC, cloud, IoT, VR/XR); **agent-based simulation** (ABMS, NetLogo).

- Roots: AI (planning → reactive (Brooks) → hybrid → BDI practical reasoning; Shoham 1993 = agent-oriented programming), OOP, HCI ("digital butlers").
- **Agents vs. objects**: objects control their state, agents also control their *behaviour* ("objects do it for free, agents do it for money") and integrate reactive, proactive and social behaviour.
- **Intentional stance** (Dennett): predict behaviour by attributing beliefs, desires, rationality. Specified with **modal logics** ($\Box$ necessity, $\Diamond$ possibility, $\Diamond p \leftrightarrow \neg\Box\neg p$).
- **Contract Net** (1983): manager announces task, contractors bid, manager awards to the best bid. **Actors** (1973): create, send, become; asynchronous message passing.

5 Agent architectures

6 Engineering multi-agent systems

Part IV

Prolog

7 Prolog

Part V

Cognitive Agents, Communication and Platforms

8 Cognitive agents: BDI and AgentSpeak(L)

9 Agent communication

10 JADE: a platform for multi-agent systems

11 Agent-based modelling and simulation